

Universidad de La Habana
Facultad de Matemática y Computación

Un lenguaje de dominio específico para la descripción declarativa de estructuras tabulares

Autor: Jose Morales Lazo

**Tutor:
MSc. Fernando Rodríguez Flores**

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

La Habana, junio 2026

“La simplicidad es el prerrequisito de la fiabilidad.”

Edsger W. Dijkstra

Agradecimientos

A mi tutor, el MSc. Fernando Rodríguez Flores, por su paciencia, sus ideas y su entusiasmo permanente ante los lenguajes y las hojas de cálculo.

A mi familia, por el apoyo incondicional a lo largo de la carrera.

A los colegas y profesores de MATCOM que convirtieron cada problema académico en una oportunidad de automatizarlo.

Opinión del Tutor

La gestión de la información en entornos académicos genera, de forma recurrente, documentos de planificación tabular: horarios de docencia, calendarios de defensas, distribuciones de aulas. Cuando se quiere automatizar la creación de esos documentos se repite invariablemente el mismo patrón: código imperativo que mezcla la lógica del problema con los detalles de la *API* de la biblioteca de salida elegida, ya sea una hoja de cálculo, una base de datos o un servicio web. El resultado es código frágil, difícil de mantener y poco reutilizable.

El trabajo que se presenta en este documento aborda precisamente ese problema. Jose Morales Lazo diseñó e implementó un lenguaje de dominio específico (DSL) interno en Common Lisp que permite declarar tablas relacionadas, columnas calculadas, reglas de visualización condicional y relaciones entre estructuras, sin hacer referencia a ningún detalle concreto del sistema de salida. El AST resultante es independiente del backend; como demostración de viabilidad, se implementó un backend que genera el programa Python que construye el archivo `.xlsx` con todas las fórmulas y reglas de formato condicional incrustadas.

El diplomante demostró durante el trabajo una notable capacidad para identificar la abstracción correcta, diseñarla y construirla con rigor. Los tres casos de uso reales validados —horario de defensas de tesis, horario docente de la facultad y horario de programación televisiva— muestran que el lenguaje es suficientemente expresivo para cubrir problemas de complejidad creciente.

Estoy muy satisfecho con los resultados obtenidos y considero que el trabajo reúne los méritos suficientes para optar al título de Licenciado en Ciencia de la Computación.

MSc. Fernando Rodríguez Flores
Facultad de Matemática y Computación
Universidad de La Habana
Junio, 2026

Resumen

Describir visualizaciones de estructuras tabulares —horarios, calendarios, distribuciones— exige, en la práctica, código imperativo que entremezcla la lógica del problema —quién tiene conflicto de horario, cuántas defensas acumula un profesor— con los detalles mecánicos del sistema de salida elegido: índices de columna, rangos absolutos, cadenas de fórmula, llamadas a la API del generador. Esta mezcla eleva el costo de desarrollo y dificulta la reutilización ante cambios en los datos o en el backend de destino.

El objetivo de este trabajo es separar ambas capas mediante un lenguaje de dominio específico (DSL) interno en Common Lisp. El programador declara *qué* debe contener la estructura —tablas relacionadas, columnas calculadas, cuantificadores existenciales, reglas de coloreado condicional, totales con posicionamiento relativo— y un compilador basado en el Sistema de Objetos de Common Lisp (CLOS) traduce esa declaración a través de un árbol de sintaxis abstracta independiente del backend. Como demostración, se implementa un backend que genera programas Python los cuales construyen archivos `.xlsx` mediante la biblioteca `openpyxl`, con reglas de formato condicional persistentes incrustadas como `FormulaRule`.

La validación sobre tres casos reales de la Facultad de Matemática y Computación de la Universidad de La Habana confirma que el lenguaje captura los patrones recurrentes de la planificación tabular con una cantidad de código significativamente menor que la alternativa imperativa, e independientemente de los detalles de cualquier API de salida.

Palabras clave: Lenguaje de dominio específico; Estructuras tabulares; Generación de código; Common Lisp; CLOS; Macros; Formato condicional; `openpyxl`.

Abstract

Describing visualizations of tabular structures—schedules, calendars, distributions—requires, in practice, imperative code that interleaves domain logic—who has a scheduling conflict, how many thesis defenses a professor is involved in—with the mechanical details of the chosen output system: column indices, absolute ranges, formula strings, and output API calls. This interleaving raises development costs and hampers reuse when data or the target backend changes.

The objective of this work is to separate these two layers through an internal domain-specific language (DSL) built in Common Lisp. The programmer declares *what* the structure should contain—related tables, computed columns, existential quantifiers, conditional coloring rules, relative totals—and a compiler based on the Common Lisp Object System (CLOS) translates that declaration through a backend-independent abstract syntax tree. As a demonstration, a backend is implemented that generates Python programs which build `.xlsx` files via the `openpyxl` library, with persistent conditional formatting rules embedded as `FormulaRule` objects.

Validation on three real use cases from the Faculty of Mathematics and Computer Science of the University of Havana confirms that the language captures the recurring patterns of tabular planning with significantly less code than the imperative alternative, independent of any specific output API.

Keywords: Domain-specific language; Tabular structures; Code generation; Common Lisp; CLOS; Macros; Conditional formatting; `openpyxl`.

Índice general

Agradecimientos	2
Opinión del Tutor	3
Resumen	4
Abstract	5
Introducción	9
1. Preliminares teóricos	12
1.1. Lenguajes de dominio específico	12
1.2. Árboles de sintaxis abstracta y generación de código	13
1.2.1. Árbol de sintaxis abstracta	13
1.2.2. Generación de código	13
1.3. El lenguaje de programación Common Lisp	14
1.3.1. Macros	14
1.3.2. Funciones genéricas y CLOS	14
1.3.3. La macro <code>defclass*</code>	15
1.4. Hojas de cálculo y el formato <code>.xlsx</code>	16
2. Fundamentos del dominio y estado del arte	17
2.1. Problemas de asignación de horarios	17
2.1.1. Horario docente universitario	18
2.1.2. Planificación de defensas de tesis	18
2.1.3. Programación televisiva	18
2.2. Metaheurísticas para la resolución de horarios	18
2.3. Estado del arte	19
2.3.1. Horario docente universitario	19
2.3.2. Programación televisiva	20
2.4. Aplicaciones existentes para la gestión de horarios	20
2.4.1. Ámbito académico	20
2.4.2. Ámbito de radiodifusión	21
2.5. Tecnologías empleadas	21
2.6. Limitaciones de los enfoques actuales	22
3. Análisis del dominio de planificación tabular	23

3.1.	Casos de uso	23
3.1.1.	Horario de defensas de tesis	23
3.1.2.	Horario docente de la facultad	24
3.1.3.	Horario de programación televisiva	24
3.2.	Elementos comunes	25
3.2.1.	Tablas relacionadas y referencias cruzadas	25
3.2.2.	Columnas calculadas	25
3.2.3.	Visualización condicional	26
3.2.4.	Posicionamiento relativo	26
3.3.	Requisitos del lenguaje	26
4.	El Lenguaje de Dominio Específico	28
4.1.	Visión general	28
4.2.	Árbol de sintaxis abstracta	29
4.2.1.	Estructura del workbook	29
4.2.2.	Expresiones de celda	29
4.2.3.	Navegación de fila y control de visibilidad	30
4.2.4.	Cuantificación existencial	30
4.2.5.	Expresiones cross-sheet dinámicas	30
4.2.6.	Formato condicional y posicionamiento	31
4.3.	Definición de tablas: def-table	31
4.4.	Expresiones de celda	32
4.5.	El cuantificador existencial exists	33
4.5.1.	Variante same-table: :all-rows	33
4.5.2.	Variante cross-table: :rows-of	33
4.6.	Formato condicional declarativo	34
4.7.	Ensamblaje del workbook	34
4.8.	Posicionamiento relativo: nav	35
4.9.	Generación y ejecución	35
4.10.	Ejemplo completo: Horario de Defensas de Tesis	36
5.	El backend Excel: generación de archivos .xlsx	42
5.1.	Arquitectura del backend	42
5.2.	Resolución de coordenadas	42
5.2.1.	Conversión de número de columna a letra Excel	42
5.2.2.	Construcción del mapa de columnas	43
5.2.3.	Cálculo de rangos de fila	43
5.3.	La función genérica compile-excel-formula	44
5.3.1.	Métodos para expresiones simples	44
5.3.2.	Métodos para rangos y agregados	45
5.4.	Generación de formato condicional	46
5.4.1.	Variante same-table: SUMPRODUCT con exclusión de fila actual	46
5.4.2.	Variante cross-table: SUMPRODUCT con COUNTIFS	46
5.5.	Generación de referencias cruzadas entre hojas	47
5.6.	Generación del programa Python	48
6.	Detalles de implementación	50
6.1.	Organización en cuatro archivos	50

6.2. Macros del DSL	50
6.2.1. Macros de expresión	50
6.2.2. Macros de estructura	52
6.2.3. El macro <code>nav</code> y el sistema de posicionamiento	53
6.3. Extensibilidad del sistema	53
6.4. Validación sobre los casos de uso	54
6.4.1. Horario de Defensas de Tesis	54
6.4.2. Horario Docente de la Facultad	54
6.4.3. Discusión comparativa	55
Conclusiones	56
Recomendaciones	58

Introducción

Contexto

La gestión de la información académica en una facultad universitaria produce, de manera recurrente, un tipo particular de documento: la hoja de cálculo de planificación. Los horarios de docencia, los calendarios de defensas de tesis, los cuadros de distribución de aulas y las programaciones de actividades extracurriculares comparten una estructura común: tablas relacionadas, con columnas que resumen información de otras tablas mediante fórmulas, y con celdas que se colorean automáticamente cuando se detecta un conflicto o una condición relevante [45].

La hoja de cálculo es la herramienta elegida para estos documentos por razones prácticas: está disponible en cualquier puesto de trabajo, no requiere instalación de software adicional y produce un resultado que el usuario final puede editar directamente. Sin embargo, cuando el documento debe generarse automáticamente a partir de datos externos —una base de datos, un formulario, un archivo JSON— la simplicidad operativa de la hoja de cálculo se convierte en complejidad de programación.

El problema

El problema central que este trabajo aborda es la mezcla de responsabilidades en el código que genera hojas de cálculo complejas.

Para ilustrarlo, consideremos un caso concreto: construir automáticamente un horario de defensas de tesis en Excel. El documento debe incluir una tabla de defensas con las columnas de rol del tribunal, una tabla de profesores con el número de defensas en que participa cada uno, y una regla de formato condicional que coloree en rojo a los profesores que aparecen en dos defensas programadas en el mismo slot horario.

Un programa imperativo que genera este documento en Python usando la biblioteca `openpyxl` debe conocer simultáneamente:

- que la columna `tutor` ocupa la letra B y `secretario` ocupa la letra F en la hoja de cálculo concreta;
- que los datos de defensas empiezan en la fila 4 y terminan en la fila 6;
- que la fórmula de conteo es `COUNTIF($B$4:$F$6,K4)`;
- que la regla de coloreado condicional es una `FormulaRule` con la expresión `SUMPRODUCT(($B$4:$B$6=$K4)+...) * COUNTIFS(...) > 0 > 0`.

Si el número de defensas cambia, o si se agrega una columna de rol, o si las tablas se reorganizan, *todos* esos valores deben recalcularse manualmente. El código no expresa la intención —“colorear a los profesores con conflicto de horario”— sino el mecanismo concreto de su implementación en una versión específica de la hoja.

Esta situación no es exclusiva del horario de defensas. El horario docente de la facultad, que distribuye grupos, turnos y aulas en múltiples hojas con referencias cruzadas, y el horario de programación televisiva, que calcula el fin de cada programa a partir de su hora de inicio y duración, presentan el mismo patrón: la lógica del problema queda ahogada entre detalles de la API.

La pregunta de investigación central de este trabajo es:

¿Puede un lenguaje de dominio específico separar la descripción semántica de una estructura tabular del mecanismo concreto de su generación, de modo que el mismo programa declarativo sea válido independientemente del backend de salida?

Hipótesis de la investigación

La utilización de un lenguaje de dominio específico (DSL) interno, implementado mediante macros de Common Lisp y un compilador basado en el Sistema de Objetos de Common Lisp (CLOS), permite separar la descripción semántica de una estructura tabular de planificación —sus tablas, sus relaciones, sus cálculos y sus condiciones de visualización condicional— de los detalles concretos de cualquier backend de generación. El backend es un componente intercambiable que traduce el árbol de sintaxis abstracta a la representación de salida deseada; el programa del DSL permanece inalterado al cambiar de backend. Esta separación reduce el código necesario, facilita la reutilización ante cambios en los datos y hace que el programa resultante sea legible por el experto del dominio sin conocimiento de ningún sistema de salida específico.

Objetivo general

Diseñar e implementar un lenguaje de dominio específico en Common Lisp para la descripción declarativa de estructuras tabulares de planificación —tablas relacionadas, columnas calculadas, cuantificación existencial y reglas de visualización condicional— con una arquitectura que separa el DSL del backend de generación, y demostrar su viabilidad mediante un backend que produce archivos `.xlsx`.

Objetivos específicos

1. Analizar las características recurrentes de las hojas de cálculo de planificación en el contexto académico, identificando los patrones comunes de estructura, cálculo y visualización condicional.
2. Diseñar la sintaxis y la semántica de un DSL declarativo capaz de expresar tablas relacionadas, referencias cruzadas entre tablas, columnas computadas, cuantificación existencial y posicionamiento relativo de expresiones auxiliares.

3. Definir un árbol de sintaxis abstracta (AST) cuyas clases representen los conceptos del dominio de manera independiente del backend de generación.
4. Implementar el DSL como un conjunto de macros de Common Lisp que construyen nodos del AST en tiempo de expansión de macros.
5. Implementar un backend de ejemplo, basado en una función genérica CLOS, que traduzca cada tipo de nodo AST a instrucciones Python para la biblioteca `openpyxl`, generando archivos `.xlsx` con formato condicional persistente.
6. Validar el lenguaje sobre tres casos de uso reales de la Facultad de Matemática y Computación de la Universidad de La Habana: el horario de defensas de tesis, el horario docente de la facultad y el horario de programación televisiva.

Estructura del documento

El documento se organiza de la siguiente manera.

El capítulo 1 sienta las bases teóricas y conceptuales del trabajo. Se presentan las definiciones fundamentales sobre hojas de cálculo, se introduce el concepto de lenguaje de dominio específico y su uso en la construcción de compiladores, se explica el papel de los árboles de sintaxis abstracta como representación intermedia, y se describen las características de Common Lisp —macros, CLOS y despacho múltiple— que hacen de él una plataforma idónea para el diseño del DSL.

El capítulo 3 analiza el dominio de la planificación tabular a partir de tres casos reales. Se identifican los elementos comunes en estos documentos y se formulan los requisitos concretos que el lenguaje debe satisfacer.

El capítulo 4 describe el lenguaje de dominio específico diseñado. Se presentan el árbol de sintaxis abstracta con sus cuatro familias de nodos, las formas del DSL visibles al programador y su semántica, y el flujo completo desde la declaración hasta la entrega del AST al backend.

El capítulo 5 describe el backend implementado como demostración: cómo traduce el AST a coordenadas del sistema de salida, cómo compila cada tipo de expresión a cadenas de fórmula, y cómo genera las fórmulas SUMPRODUCT que implementan el formato condicional persistente en los archivos `.xlsx`.

El capítulo 6 describe los detalles de implementación: la organización en cuatro archivos con responsabilidades separadas, la macro `defclass*` que automatiza la definición de nodos del AST, y los principios de extensibilidad del sistema.

El documento concluye con las conclusiones del trabajo, las recomendaciones para investigaciones futuras y las referencias bibliográficas consultadas.

Capítulo 1

Preliminares teóricos

Este capítulo presenta los fundamentos teóricos necesarios para el desarrollo del DSL descrito en este trabajo. La sección 1.1 introduce los lenguajes de dominio específico y su clasificación. La sección 1.2 describe los árboles de sintaxis abstracta y el proceso de generación de código. La sección 1.3 detalla las características de Common Lisp que lo convierten en la plataforma de implementación elegida. La sección 1.4 define los términos necesarios para comprender el backend de demostración implementado, que produce archivos `.xlsx`.

1.1 Lenguajes de dominio específico

Los lenguajes de programación de propósito general (GPL, del inglés *General-Purpose Language*) como C, Java o Python ofrecen flexibilidad máxima, pero a costa de mayor verbosidad y menor expresividad dentro de dominios concretos. Un **lenguaje de dominio específico** (*Domain-Specific Language*, DSL) es un lenguaje diseñado para expresar soluciones dentro de un dominio particular de problemas, ofreciendo abstracciones que reflejan directamente los conceptos y patrones propios de ese dominio [1].

Definición 1.1 (DSL, [9]). *Un lenguaje de dominio específico es un lenguaje de programación o especificación ejecutable que ofrece, mediante abstracciones apropiadas, poder expresivo centrado en —y normalmente restringido a— un dominio de problemas particular.*

Los DSL se clasifican según su relación con el lenguaje anfitrión en dos grandes tipos [1, 10]:

DSL interno (embebido). Se construye usando la sintaxis y las capacidades del lenguaje anfitrión. No requiere herramientas de análisis léxico ni sintáctico propias: el programador usa el lenguaje anfitrión como plataforma y el DSL actúa como una biblioteca con vocabulario de dominio. Common Lisp es especialmente adecuado para este enfoque gracias a su sistema de macros, que permite extender la sintaxis del lenguaje sin necesidad de un compilador externo.

DSL externo. Define su propia sintaxis y requiere un lexer y un parser propios. El DSL es un lenguaje independiente del anfitrión; ejemplos conocidos son SQL,

L^AT_EX y Matlab. Este enfoque ofrece mayor libertad sintáctica pero implica un esfuerzo de implementación significativamente mayor.

El DSL desarrollado en este trabajo es **interno**: está escrito en Common Lisp, sus formas son expresiones Lisp válidas y no requiere ningún parser externo. Los macros de Lisp actúan como el compilador del DSL, expandiendo las formas del dominio en constructores de nodos del AST.

1.2 Árboles de sintaxis abstracta y generación de código

1.2.1 Árbol de sintaxis abstracta

Un **árbol de sintaxis abstracta** (AST, del inglés *Abstract Syntax Tree*) es una representación estructurada en forma de árbol del código fuente de un programa [2]. A diferencia del árbol sintáctico concreto (CST), el AST abstrae los detalles de la sintaxis concreta —paréntesis, palabras clave, signos de puntuación— y retiene únicamente la información semántica relevante: la estructura lógica del programa [48].

Cada nodo del árbol corresponde a un constructo del lenguaje y sus hijos representan los subcomponentes de ese constructo. Por ejemplo, la expresión `a + b` se representa como un nodo de suma con dos hijos: el nodo del identificador `a` y el nodo del identificador `b`.

En el diseño del DSL presentado en este trabajo, el AST cumple el papel central de **representación intermedia independiente del backend**. El programador construye el AST mediante las formas del DSL; un backend lo recorre y produce la salida deseada. La elección del backend —hojas de cálculo, bases de datos, una API web, un documento HTML— es ortogonal al programa DSL: el mismo AST alimenta cualquier backend que implemente la interfaz de compilación.

1.2.2 Generación de código

La **generación de código** es el proceso que transforma el AST en código ejecutable o en código fuente de otro lenguaje, preservando el comportamiento descrito en el programa original [2]. El generador recorre el árbol nodo a nodo, produciendo para cada nodo la representación correspondiente en el lenguaje de destino.

Este proceso puede producir código en cualquier lenguaje destino, lo que hace al AST una representación intermedia que desacopla el lenguaje de descripción del lenguaje de ejecución. En este trabajo, el backend implementado genera programas Python para la biblioteca `openpyxl`; sin embargo, la misma representación AST podría alimentar un generador alternativo que produzca, por ejemplo, sentencias SQL o documentos HTML.

Cada nodo del AST en este trabajo es una instancia de una clase CLOS definida con la macro `defclass*`. Los nodos se dividen en seis familias:

1. Nodos de estructura del workbook (`xl-workbook`, `xl-sheet`, `xl-region`, `xl-table`).
2. Nodos de expresión de celda (`xl-expr-column-ref`, `xl-table-range`, `xl-expr-countif`, `xl-expr-if`, etc.).
3. Nodos de navegación por filas (`xl-expr-previous-row`, `xl-expr-next-row`, `xl-expr-first-row`, etc.).

4. Nodos de cuantificación existencial (`xl-expr-exists`, `xl-domain-rows`).
5. Nodos de referencia cruzada entre hojas (`xl-expr-cross-sheet-ref`, `xl-expr-cross-cell`, `xl-expr-collect-over`).
6. Nodos de formato condicional y posicionamiento (`xl-style-rule`, `xl-anchor`, `xl-nav`, `xl-sheet-fixed-expr`).

Esta organización se describe en detalle en el capítulo 4.

1.3 El lenguaje de programación Common Lisp

Common Lisp es un lenguaje de programación multiparadigma que combina los estilos funcional, imperativo y orientado a objetos [3]. Fue diseñado originalmente por John McCarthy en 1958 [11] y estandarizado como Common Lisp en 1994. Su elección como plataforma de implementación del DSL se justifica por su homoiconicidad y por dos mecanismos fundamentales: los macros y las funciones genéricas.

Homoiconicidad: el código comparte la misma estructura que los datos. En Common Lisp, todas las expresiones son listas anidadas con notación prefija: el primer elemento es el operador y los siguientes son los argumentos. Una llamada a función, la definición de una clase, una iteración y una forma del DSL tienen exactamente la misma estructura sintáctica. Esta propiedad hace que el compilador pueda inspeccionar y transformar el código del mismo modo que manipula listas de datos ordinarias [4]. Los DSL internos se benefician directamente de esto: las formas del DSL son listas ordinarias que los macros transforman en código válido sin necesidad de ningún parser externo.

1.3.1 Macros

Los **macros** de Common Lisp implementan metaprogramación: operan sobre representaciones simbólicas del código fuente antes de su evaluación, extendiendo el lenguaje sintácticamente [5]. Un macro recibe como argumento el código fuente —representado como una lista— y devuelve código nuevo que sustituye a la forma original antes de la compilación.

La diferencia con una función ordinaria es temporal: una función se evalúa en tiempo de ejecución; un macro se expande en tiempo de compilación. Esta distinción permite introducir nueva sintaxis y nuevas abstracciones sin coste adicional en tiempo de ejecución. Es esencial para implementar DSL internos que se leen como vocabulario del dominio pero se compilan a estructuras de datos eficientes.

En este trabajo, cada constructo del DSL —`def-table`, `conditional-rendering`, `exists`, `nav`— se implementa como un macro que construye uno o más nodos del AST. El programador escribe en vocabulario del dominio; el macro transforma esa escritura en llamadas a los constructores CLOS.

1.3.2 Funciones genéricas y CLOS

El **Sistema de Objetos de Common Lisp** (CLOS, del inglés *Common Lisp Object System*) introduce las **funciones genéricas**: una interfaz común con múltiples métodos especializados según las características de los argumentos [12]. El despacho dinámico se

realiza en función del tipo de los argumentos en tiempo de ejecución, lo que separa la definición de la interfaz de sus implementaciones concretas.

Las funciones polimórficas se declaran con `defgeneric` y se especializan con `defmethod`. A diferencia de los sistemas de objetos convencionales, CLOS soporta **despacho múltiple**: el método seleccionado puede depender del tipo de varios argumentos simultáneamente, no solo del “receptor” [13].

En este trabajo, CLOS cumple dos funciones. Primero, las clases CLOS representan los nodos del AST: cada tipo de nodo es una clase con slots que almacenan sus atributos. Segundo, la función genérica del backend —`compile-excel-formula` en la implementación de demostración— usa el despacho múltiple para compilar cada tipo de nodo a la representación de salida correspondiente, sin ninguna estructura `cond` o `case` centralizada. Añadir soporte para un nuevo tipo de nodo solo requiere definir un nuevo `defmethod`; el resto del sistema no se modifica.

1.3.3 La macro `defclass*`

La notación prefija de Lisp y la ausencia de la distinción entre código y datos hacen innecesario un lexer externo para construir el AST del DSL: cada forma del DSL es directamente una lista anidada que un macro puede transformar en un nodo del AST. Para reducir el código repetitivo de definir clases CLOS y sus constructores, el sistema define la macro `defclass*`, que genera simultáneamente:

- Una clase CLOS con nombre `class-xl-nombre`, donde cada slot tiene un accessor y un `initarg` del mismo nombre.
- Una función constructora `xl-nombre` con argumentos por clave (`&key`), que crea una instancia de la clase con los valores proporcionados.

Listing 1.1: Implementación de `defclass*`.

```
(defmacro defclass* (class-name parents slots
                    &key (ctr-args-type '&key))
  `(progn
    (defclass ,(symp 'class- class-name) ,parents
      ,(loop for s in slots
        collecting (expansion-del-slot s)))
    ,(make-constructor class-name slots
      :argument-type ctr-args-type)))
```

Por ejemplo, la definición del nodo `xl-expr-if`:

Listing 1.2: Ejemplo de uso de `defclass*`.

```
(defclass* xl-expr-if () (test then else))
;; genera:
;; (defclass class-xl-expr-if ()
;;   ((test :accessor test :initarg :test)
;;    (then :accessor then :initarg :then)
;;    (else :accessor else :initarg :else)))
;; (defun xl-expr-if (&key test then else)
```



```
;;      (make-instance 'clase-xl-expr-if  
;;      :test test :then then :else else))
```

Este patrón elimina el código repetitivo de definición de nodos y garantiza que cada clase tiene una función constructora consistente con su interfaz de inicialización.

1.4 Hojas de cálculo y el formato `.xlsx`

El backend implementado en este trabajo produce archivos `.xlsx`, el formato de Microsoft Excel basado en Office Open XML [8]. Los conceptos siguientes son necesarios para comprender ese backend; no son centrales al diseño del DSL, que es independiente de cualquier formato de salida.

Una **hoja de cálculo** (*spreadsheet*) es una aplicación para organizar, calcular y visualizar datos en una cuadrícula bidimensional de celdas. Cada celda puede contener un valor literal —un número, una fecha o una cadena de texto— o una **fórmula**: una expresión cuyo resultado se recalcula automáticamente cuando cambia cualquier celda de la que depende [14].

Workbook (libro de trabajo). Un archivo `.xlsx` que agrupa una o más hojas de trabajo. Es la unidad de intercambio del formato Office Open XML [8].

Hoja de trabajo. Una cuadrícula de celdas dentro del workbook, identificada por un nombre (p. ej., *Defensas*).

Celda. La unidad mínima de almacenamiento, referenciada por una dirección del tipo B4 (columna B, fila 4) o `$B$4` (referencia absoluta).

Fórmula. Una expresión que comienza con `=` y puede contener referencias a otras celdas, rangos y funciones predefinidas como `COUNTIF`, `COUNTA`, `SUM`, `IF` o `SUMPRODUCT`.

Rango. Un bloque rectangular de celdas contiguas, expresado como `$B$4:$F$6` (columnas B–F, filas 4–6).

Formato condicional. Un mecanismo que aplica estilos visuales —color de relleno, color de texto, borde— a las celdas de un rango cuando se cumple una condición evaluada en tiempo de apertura o edición del archivo. Una **FormulaRule** es el tipo más general de regla de formato condicional: la condición es una fórmula arbitraria, lo que permite expresar condiciones que involucran otras hojas, otras celdas o funciones de agregación [7].

La biblioteca **openpyxl** es la herramienta Python elegida para producir archivos `.xlsx` en este trabajo. Permite crear workbooks, escribir valores y fórmulas en celdas, y definir **FormulaRule** que quedan incrustadas en el archivo y se evalúan en Excel o LibreOffice Calc sin necesidad de código adicional [7].

Capítulo 2

Fundamentos del dominio y estado del arte

La asignación de horarios es un área de estudio dentro de la planificación de recursos y la optimización que abarca múltiples contextos prácticos [15]. Este capítulo establece las bases conceptuales del dominio y analiza el panorama de soluciones existentes. La sección 2.1 define los tipos de problemas de asignación de horarios y sus variantes representativas. La sección 2.3 presenta el estado del arte en ambos dominios. Las secciones 2.4 y 2.5 examinan las herramientas comerciales disponibles y las tecnologías sobre las que se apoyan. Finalmente, la sección 2.6 identifica las limitaciones que motivan el trabajo presentado en esta tesis.

2.1 Problemas de asignación de horarios

La asignación de horarios constituye un problema de planificación presente en muy diversos contextos [15]. Consiste en organizar y distribuir un conjunto de actividades en determinados intervalos de tiempo, bajo un conjunto de restricciones que limitan las posibles asignaciones.

Estas restricciones se clasifican en dos tipos. Las **restricciones duras** son obligatorias: si no se cumplen, la solución no es válida —dos clases no pueden compartir el mismo aula a la misma hora; un profesor no puede impartir dos asignaturas simultáneamente—. Las **restricciones blandas** no invalidan la solución pero afectan su calidad: concentrar todas las clases de una asignatura en un solo día o colocar una materia de alta carga cognitiva en el último turno de la tarde son situaciones admisibles pero no deseables.

Entre las restricciones más comunes se distinguen los **conflictos** y los **solapamientos**. Un conflicto ocurre cuando dos o más actividades compiten por el uso simultáneo de un mismo recurso limitado —un aula, un profesor, un canal de emisión—. Un solapamiento se produce cuando actividades asignadas coinciden en el tiempo de manera incompatible, impidiendo su ejecución simultánea por parte de los actores involucrados.

Además de estas restricciones fundamentales, es necesario gestionar la disponibilidad real de espacios y personas, lo que incrementa la complejidad del problema y restringe aún más el conjunto de soluciones viables. El problema varía según el contexto: cada

entorno tiene sus propias reglas y objetivos. Las subsecciones siguientes describen las variantes más relevantes para este trabajo.

2.1.1 Horario docente universitario

Este tipo de problema consiste en asignar cursos, profesores y aulas a determinados períodos de tiempo cumpliendo múltiples restricciones [16, 17, 15]: la disponibilidad de los docentes, la capacidad de las aulas y la no superposición de materias para un mismo grupo de estudiantes. El objetivo principal es generar un horario factible que optimice el uso de los recursos y minimice conflictos.

Por ejemplo, al programar la asignatura *Álgebra* para el grupo C111 los lunes a las 9:15 h en el aula 5, el sistema debe asegurar que el profesor asignado no tenga otra clase a esa misma hora, que el aula 5 no esté ocupada por otro grupo, y que el grupo C111 no tenga otra asignatura en ese turno. Las restricciones concretas varían entre instituciones: en algunas la escasez de aulas es la restricción dominante; en otras, los laboratorios especializados crean los cuellos de botella más severos.

2.1.2 Planificación de defensas de tesis

La planificación de defensas de tesis es una variante del problema de asignación de horarios con características propias. Cada defensa exige la presencia simultánea de varios miembros de tribunal: tutor, oponente, presidente, vocal y secretario. La restricción dura crítica es la ausencia de conflictos: un mismo miembro del tribunal no puede estar en dos defensas programadas en el mismo slot horario.

A diferencia del horario docente, en este caso el recurso escaso no son las aulas sino los profesores con el perfil adecuado para cada rol [40]. Detectar estos conflictos de forma visual es precisamente uno de los requisitos que motivaron el desarrollo del DSL descrito en este trabajo.

2.1.3 Programación televisiva

En la confección de la parrilla televisiva, el reto es organizar los programas en franjas horarias para maximizar la audiencia y cumplir los compromisos comerciales [18]. Se consideran factores como el tipo de público en cada franja, la competencia con otros canales y la duración exacta de los programas [19].

El programador debe, por ejemplo, evitar que un noticiero importante coincida con la transmisión en vivo de un evento deportivo de duración variable, o garantizar que un bloque infantil se emita en las mañanas de fin de semana y no en la franja nocturna. A diferencia del entorno académico, el enfoque aquí es estratégico y comercial: las restricciones blandas —cómo posicionar los programas para maximizar la cuota de pantalla— tienen tanto peso como las restricciones duras.

2.2 Estado del arte

2.2.1 Horario docente universitario

La investigación sobre generación automática de horarios universitarios tiene décadas de historia. Abdennadher y Marte [16] formalizaron el problema mediante Reglas de Manejo de Restricciones (CHR), estableciendo un lenguaje declarativo de restricciones que influyó en trabajos posteriores. Müller [17] desarrolló un sistema completo de timetabling basado en satisfacción de restricciones para entornos universitarios. Reddy et al. [42] propusieron un modelo de coloración de grafos que garantiza formalmente la ausencia de conflictos pero resulta computacionalmente costoso en instancias reales.

En la Facultad de Matemática y Computación de la Universidad de La Habana (MAT-COM/UH) existe una línea de investigación consolidada sobre este problema. Portela Aguilera [33] realizó la primera aproximación formal a la generación automática de horarios para la facultad, sentando las bases del problema en el contexto local. Travieso Sosa [34] formalizó el conjunto de restricciones del problema e introdujo el concepto de “felicidad” como medida ponderada de la calidad del horario. Chang Alcover [35] aplicó satisfacción de restricciones (CSP) con retroceso y búsqueda local. Fernández Llanos [36] desarrolló una API especializada con evaluación incremental de restricciones.

Seguí Bonilla [37] y Tomás López [38] desarrollaron módulos de búsqueda automática de horarios basados en técnicas de optimización computacional. Quintero Rojas [39] integró los módulos anteriores en un sistema completo de gestión de horarios. Nardo González [40] extendió la investigación al problema de planificación de defensas de tesis, que comparte la estructura de conflictos del horario docente pero con restricciones específicas sobre los miembros de tribunal. Casteleiro Wong [41] abordó la informatización de la gestión docente en el contexto de la facultad.

2.2.2 Programación televisiva

En el ámbito de la radiodifusión, Bollapragada et al. [18] plantearon el problema de la programación televisiva como programación lineal entera mixta (MILP) orientada a maximizar la audiencia en cadenas de televisión comerciales. Fernández y Salido [19] lo modelaron como un CSP y propusieron técnicas de descomposición del espacio de búsqueda junto con métodos de búsqueda automática.

A partir de 2015 la incorporación de técnicas de aprendizaje automático marcó una nueva dirección. Kim et al. [43] propusieron un sistema de aprendizaje por refuerzo profundo para la programación televisiva, publicado en *IEEE Transactions on Broadcasting*.

En ambos dominios la tendencia reciente es hacia herramientas interactivas que asisten al experto sin reemplazarlo: el planificador mantiene el control creativo mientras el sistema garantiza la viabilidad técnica frente a las restricciones.

2.3 Aplicaciones existentes para la gestión de horarios

Dada la complejidad y la gran cantidad de variables de estos problemas, confeccionar horarios de forma manual en herramientas básicas resulta un proceso lento y propenso a errores humanos [20]. Por este motivo existen aplicaciones especializadas, tanto en el ámbito académico como en el de la radiodifusión. Estas aplicaciones van desde soluciones

de escritorio hasta plataformas de software como servicio (*Software as a Service*, SaaS) accesibles desde el navegador.

2.3.1 Ámbito académico

En la gestión del *Educational Timetabling Problem* (ETP) [15] destacan las siguientes herramientas:

Coursedog y TimeEdit. Representan el estado del arte en educación superior [21]. Son plataformas SaaS que permiten a coordinadores y decanos gestionar recursos en tiempo real, con interfaces de validación que alertan visualmente sobre conflictos de aula o solapamientos de profesores.

aSc Horarios. Una de las herramientas más extendidas para educación primaria y secundaria [20]. Destaca por su capacidad de generar horarios automáticos basados en heurísticas y por su interfaz interactiva de arrastrar y soltar que facilita la edición manual asistida.

Syllabus Plus. Utilizada por grandes universidades, se centra en la optimización del horario desde la perspectiva del estudiante: garantiza que el calendario sea viable para el cuerpo estudiantil y no solo para el cuerpo docente.

Horarium.ai. Plataforma de generación automática e interactiva de horarios mediante inteligencia artificial [44]. Incorpora modelos de lenguaje para asistir al planificador en la toma de decisiones durante la confección del horario.

2.3.2 Ámbito de radiodifusión

En televisión y radiodifusión, las aplicaciones deben manejar la precisión temporal de la emisión junto con las restricciones comerciales:

MEDIAGENIX (WHATS'On). Estándar europeo para la gestión de canales de televisión y plataformas VOD. Permite la planificación de parrillas a largo plazo, validando automáticamente los derechos de emisión y las reglas de pauta publicitaria [22].

BroadView y Amagi. Soluciones que integran la planificación del tráfico de emisión con la automatización del *layout*. Aseguran que la lista de programación se ejecute sin errores técnicos ni huecos de emisión (*blackouts*) [23].

2.4 Tecnologías empleadas

La arquitectura tecnológica de estas herramientas ha evolucionado hacia ecosistemas en capas que separan la interfaz de usuario, el motor de optimización y el almacenamiento:

Interfaces de usuario. La mayoría de las aplicaciones modernas utilizan *frameworks* de JavaScript como React.js, Vue.js o Angular [24]. Estas tecnologías permiten crear interfaces reactivas donde el usuario recibe retroalimentación inmediata —cambios de color, alertas— ante el incumplimiento de una restricción.

Backend y motores de optimización. El procesamiento intensivo se delega a servicios escritos en lenguajes de alto rendimiento como Java, Go o Python. Estos

servicios ejecutan motores de satisfacción de restricciones (*Constraint Solvers*) como OptaPlanner o Google OR-Tools, que operan de forma independiente a la interfaz de usuario.

Persistencia. Las aplicaciones utilizan bases de datos relacionales como PostgreSQL o SQL Server para gestionar la concurrencia cuando varios usuarios editan el horario simultáneamente y para mantener un historial de auditoría de los cambios.

Formatos de intercambio. Para la exportación de datos, los sistemas académicos emplean estándares como JSON e iCal (.ics). En el ámbito televisivo, el estándar predominante es el BXF (*Broadcast eXchange Format*), un esquema XML que permite la comunicación entre el software de programación y los equipos de emisión física [25]. En entornos institucionales más modestos, el formato de intercambio más frecuente es la hoja de cálculo: un archivo `.xlsx` que concentra toda la información del horario y que el personal puede editar sin software especializado.

2.5 Limitaciones de los enfoques actuales

Los sistemas descritos resuelven bien el problema de *asignación*: dado un conjunto de recursos, actividades y restricciones, encontrar una distribución factible o cercana al óptimo. Sin embargo, presentan una limitación estructural cuando el requisito es producir una *visualización* del horario resultante: la representación visual queda acoplada al sistema que la generó.

En entornos donde la herramienta de visualización preferida es la hoja de cálculo —porque es accesible sin instalación, editable por el usuario y fácilmente compartible— la generación automática de ese documento exige código que entremezcla la lógica del problema con los detalles de la API de la biblioteca de salida. Esta mezcla, descrita en detalle en la introducción, es la motivación central del lenguaje de dominio específico desarrollado en este trabajo.

Capítulo 3

Análisis del dominio de planificación tabular

Para diseñar un lenguaje que capture los patrones recurrentes de la planificación tabular, es necesario estudiar primero los casos concretos que el lenguaje debe resolver. En este capítulo se presentan tres casos de uso reales de la Facultad de Matemática y Computación de la Universidad de La Habana, se identifican los elementos comunes entre ellos y se derivan los requisitos del lenguaje.

3.1 Casos de uso

3.1.1 Horario de defensas de tesis

El proceso de planificación de defensas de tesis de diploma asigna a cada estudiante un conjunto de cinco profesores que forman el tribunal —tutor, oponente, presidente, vocal y secretario—, una fecha y hora de exposición, y un local donde esta se celebra. El coordinador de defensas debe detectar conflictos: un mismo profesor asignado a dos defensas con el mismo slot horario (mismo día y hora), lo que hace imposible su presencia en ambas.

La hoja de cálculo que da soporte a este proceso contiene dos tablas:

Tabla de defensas. Una fila por defensa, con columnas para el estudiante y los cinco roles del tribunal, más las columnas de día, hora y local. Esta tabla no necesita columnas calculadas: toda la información se introduce manualmente.

Tabla de profesores. Una fila por profesor, con su nombre, grado académico y el número de defensas en que participa. Este último dato se calcula automáticamente contando cuántas veces aparece el nombre del profesor en cualquiera de los cinco roles de la tabla de defensas. Además, las filas de los profesores con conflicto de horario se resaltan en rojo.

Los datos del caso de validación incluyen tres defensas y seis profesores. Dos de las defensas comparten el slot **Lunes 10:00** con integrantes de tribunal en común, lo que genera un conflicto que la hoja debe detectar y resaltar:

Las defensas de Juan Díaz y Luis Mora comparten el slot **Lunes 10:00**. El profesor

Cuadro 3.1: Datos de las defensas del caso de validación.

Estudiante	Tutor	Opon.	Pres.	Vocal	Secr.	Día	Hora
Juan Díaz	Piad	García	López	Torres	Ruiz	Lunes	10:00
María Vega	García	López	Piad	Soto	Torres	Lunes	14:00
Luis Mora	López	Torres	García	Ruiz	Soto	Lunes	10:00

López participa como oponente en la primera y como tutor en la segunda; el profesor Torres como vocal en la primera y como oponente en la segunda. Ambos tienen conflicto y deben aparecer resaltados en rojo en la tabla de profesores.

Este caso es el que se usa como ejemplo motivador a lo largo del capítulo 4.

3.1.2 Horario docente de la facultad

La asignación de docencia en la Facultad de Matemática y Computación implica distribuir los turnos de clase de cada semana entre los grupos de las tres carreras (Ciencia de la Computación, Matemática y Ciencia de Datos), en cuatro años cada una. Cada grupo tiene una hoja propia en el workbook con su horario semanal.

La hoja de cálculo de planificación contiene, para cada grupo, tres tablas dispuestas en la misma fila de la hoja:

Tabla de turnos (`turno-table`). Seis columnas (turno, lunes, martes, miércoles, jueves y viernes), con dos filas Excel por turno lógico (`cell-height = 2`): la primera subfila almacena el nombre de la asignatura y la segunda el aula asignada.

Tabla de estadísticas (`stats-table`). Una fila por asignatura, con su abreviatura, nombre completo, frecuencia semanal requerida, asignaciones ya realizadas (calculadas contando las apariciones de la abreviatura en la tabla de turnos) y horas que faltan por asignar (calculadas como la diferencia entre frecuencia y asignadas).

Tabla de aulas (`aulas-table`). Lista de aulas disponibles para el grupo.

Además de las hojas de grupo, existe una hoja de ocupación de aulas que agrega, para cada turno de cada día, los grupos que ocupan cada aula. Esta tabla usa referencias cruzadas dinámicas (`collect-over`) que iteran sobre todas las hojas de grupo y extraen el aula asignada en cada turno. Diecisiete grupos generan diecisiete hojas y una hoja de aulas con referencias a las diecisiete.

Este caso pone a prueba dos características del DSL que el caso de defensas no ejercita: las tablas con celdas de altura compuesta (`cell-height` mayor que 1) y las referencias cruzadas entre hojas del mismo workbook mediante `collect-over`.

3.1.3 Horario de programación televisiva

El tercer caso de uso es el horario de programación semanal de un canal de televisión. Cada programa tiene una hora de inicio, una duración en minutos, un nombre, un tipo y un público objetivo. La hoja de cálculo muestra una página por día, con columnas para

cada uno de esos campos más una columna de hora de fin, calculada automáticamente a partir de la hora de inicio y la duración.

La fórmula de la hora de fin involucra operaciones de tiempo:

$$\text{fin} = \text{TEXT}(\text{TIMEVALUE}(\text{inicio}) + \text{duración}/1440, "hh:mm")$$

donde 1440 es el número de minutos en un día. Además, cada fila se colorea según el tipo de programa (informativo, musical, infantil, etc.) mediante un color distinto en la celda.

Este caso valida que el DSL puede expresar operaciones aritméticas sobre valores de tiempo (`time-add`) y fórmulas de celda simples sin cuantificadores existenciales.

3.2 Elementos comunes

Del análisis de los tres casos se identifican cuatro patrones que se repiten en todos o en la mayoría de los documentos de planificación tabular:

3.2.1 Tablas relacionadas y referencias cruzadas

En todos los casos, el documento contiene más de una tabla cuyos datos están relacionados. La tabla de profesores referencia la tabla de defensas; la tabla de estadísticas referencia la tabla de turnos; la hoja de aulas referencia las hojas de cada grupo.

Estas referencias son de dos tipos:

- **Intrarregión:** ambas tablas están en la misma hoja y se accede a ellas mediante rangos absolutos de columnas (p. ej., `$B$4:$F$6` para los cinco roles de la tabla de defensas).
- **Interhoja:** la tabla referenciada está en otra hoja del mismo workbook (p. ej., la hoja de aulas referencia las hojas de grupos).

El lenguaje debe resolver estas referencias sin que el programador escriba letras de columna ni nombres de hoja: los identifica por los símbolos DSL de las columnas y por el identificador de la tabla.

3.2.2 Columnas calculadas

En todos los casos, al menos una columna de alguna tabla muestra un valor calculado a partir de datos de otra tabla. Los tipos de cálculo más frecuentes son:

COUNTIF. Cuenta cuántas celdas de un rango cumplen un criterio. Ejemplo: número de veces que aparece el nombre de un profesor en los cinco roles de la tabla de defensas.

COUNTA. Cuenta celdas no vacías en un rango. Ejemplo: número de defensas registradas.

SUM. Suma un rango. Ejemplo: total de defensas acumuladas por todos los profesores.

Aritmética de tiempo. Suma una duración en minutos a una hora de inicio. Ejemplo: hora de fin de un programa televisivo.

collect-over. Agrega resultados de la misma posición en múltiples hojas. Ejemplo: grupos que ocupan el Aula 1 en el turno 3 del lunes.

3.2.3 Visualización condicional

En los casos de defensas y de docencia, la condición de coloreado depende de datos de otra tabla. El color no es estático: si el usuario edita los datos directamente en Excel, el coloreado debe recalcularse automáticamente.

El mecanismo adecuado es, por tanto, una **FormulaRule** de formato condicional incrustada en el archivo **.xlsx**, no un coloreado estático aplicado en tiempo de generación. La condición de la regla es una fórmula **SUMPRODUCT** que captura la semántica del cuantificador existencial del DSL.

La forma en que el DSL expresa esta condición —mediante **exists**— y la forma en que el backend la implementa —mediante **SUMPRODUCT**— se detallan en los capítulos 4 y 5, respectivamente.

3.2.4 Posicionamiento relativo

Todos los casos requieren expresiones fuera de las tablas: totales, etiquetas, celdas de resumen. La posición de estas expresiones depende de cuántas filas de datos tiene la tabla, que varía en cada ejecución.

La solución adoptada en el DSL es el sistema de anclas y navegación (**nav**, **ultima-fila**, **primera-fila**): se declara que una expresión va “una fila por debajo de la última fila de datos de la columna **estudiante** de la tabla de defensas”, y el compilador resuelve esa declaración a la coordenada Excel concreta en tiempo de generación.

3.3 Requisitos del lenguaje

Del análisis anterior se derivan los requisitos que el DSL debe satisfacer:

1. **Declaración de tablas.** El lenguaje debe permitir definir tablas con nombre, con una lista de columnas identificadas por símbolos, y con soporte para celdas de altura y anchura compuesta.
2. **Columnas computadas.** Debe ser posible asociar una expresión del DSL a una columna; el compilador genera la fórmula Excel correspondiente para cada fila de datos.
3. **Referencias intrarregión.** El lenguaje debe resolver referencias a columnas de la misma tabla (**col**) y a rangos de columnas de otra tabla en la misma región (**trange**, **tcol**).
4. **Cuantificación existencial.** El lenguaje debe ofrecer una forma de expresar condiciones de existencia sobre filas de la misma tabla (**:all-rows**) o de otra tabla (**:rows-of**), especificando columnas de coincidencia y columnas de solapamiento.
5. **Formato condicional persistente.** Las reglas de coloreado condicional deben traducirse a **FormulaRule** en el **.xlsx**, de modo que persistan y se reevalúen cuando el usuario edita el archivo.

6. **Posicionamiento relativo.** Debe ser posible colocar expresiones fijas en posiciones calculadas como un ancla más pasos direccionales, sin coordenadas absolutas codificadas.
7. **Referencias interhoja.** El lenguaje debe soportar referencias dinámicas a celdas de otras hojas mediante iteración sobre una lista de nombres de hojas (`collect-over`).
8. **Ensamblaje declarativo.** El workbook completo debe poderse describir con formas del DSL (`libro`, `hoja`, `tabla`) sin ningún conocimiento de las coordenadas del archivo de salida.
9. **Independencia del backend.** El AST no debe contener ningún detalle del target de generación. Agregar un nuevo tipo de expresión no debe requerir modificar el DSL; solo añadir un nuevo método al backend.

La tabla 3.2 muestra cómo cada requisito se manifiesta en los casos de uso analizados.

Cuadro 3.2: Requisitos del lenguaje y casos de uso en que se manifiestan.

Requisito	Defensas	Facultad	TV
Declaración de tablas	✓	✓	✓
Columnas computadas	✓	✓	✓
Referencias intrarregión	✓	✓	—
Cuantificación existencial	✓	—	—
Formato condicional persistente	✓	—	—
Posicionamiento relativo	✓	✓	—
Referencias interhoja	—	✓	—
Ensamblaje declarativo	✓	✓	✓
Independencia del backend	✓	✓	✓

Capítulo 4

El Lenguaje de Dominio Específico

4.1 Visión general

Numerosos problemas cotidianos producen información que se organiza naturalmente en forma de tabla: filas que representan entidades y columnas que describen sus atributos, relaciones o restricciones. Los horarios son un ejemplo paradigmático de esta clase de datos—francas horarias, participantes, recursos y conflictos pueden expresarse como tablas relacionadas—pero la misma estructura aparece en inventarios, resultados académicos, planificación de proyectos y cualquier dominio donde la información admita una organización tabular.

El objetivo del lenguaje de dominio específico (DSL) desarrollado en este trabajo es ofrecer un recurso de descripción para esa clase de datos: un vocabulario declarativo con el que el programador expresa *qué* contiene cada tabla—columnas, fórmulas derivadas, relaciones entre tablas, reglas de presentación—sin ocuparse de *cómo* se materializa esa descripción en un formato concreto.

El DSL está implementado como un conjunto de funciones y macros de Common Lisp que construyen, en tiempo de carga, un *árbol de sintaxis abstracta* (AST) cuyos nodos son instancias de clases CLOS. La canalización de compilación comprende tres etapas claramente separadas:

1. **Expansión de macros.** Cada forma del DSL (p.ej. `def-table`, `exists`) se expande en una llamada al constructor CLOS del nodo AST correspondiente. El resultado es un árbol cuyos nodos representan las entidades a partir de las cuales el sistema es capaz de generar las representaciones visuales de los datos.
2. **Generación de código.** El árbol se recorre por el backend, que traduce cada nodo a las instrucciones propias del formato de salida: expresiones de celda, reglas de presentación y directivas de construcción del documento. El backend es el único punto que conoce los nombres de columna reales, los índices de fila y los detalles de la API de destino.
3. **Ejecución.** El programa generado se ejecuta, produciendo el archivo de salida con los datos, las fórmulas derivadas y las reglas de presentación incrustadas.

Un principio central de diseño es la *separación entre intención e implementación*: el

DSL expresa la semántica del problema (“resaltar los profesores que tienen un conflicto de horario”) y el backend decide el mecanismo concreto según el formato de destino elegido. El programador del DSL no escribe ninguna instrucción específica del formato de salida.

4.2 Árbol de sintaxis abstracta

El árbol de sintaxis abstracta (AST) es la representación intermedia que separa la capa del DSL de la capa del generador de código. Cada concepto del problema—una tabla, una expresión de celda, un cuantificador existencial—queda capturado como un nodo AST con atributos propios. El backend opera únicamente sobre este árbol; no interpreta texto ni inspecciona las formas del DSL.

Todos los nodos del AST se definen con la macro `defclass*`, que genera simultáneamente una clase CLOS `clase-xl-nombre` y una función constructora `xl-nombre` con parámetros por clave. Por ejemplo, el nodo del cuantificador existencial se define y construye de la siguiente manera:

Listing 4.1: Patrón de definición de nodo AST con `defclass*`.

```
;; defclass* genera la clase y el constructor xl-expr-exists:
(defclass* xl-expr-exists ()
  (bind-var domain match-keys overlap-cols self-in-
    cols))

;; El constructor acepta los slots como :keywords:
(xl-expr-exists :bind-var      'r
                :domain        (xl-domain-rows :table-id nil)
                :match-keys     '(dia hora)
                :overlap-cols   '(tutor oponente presidente
                                vocal secretario)
                :self-in-cols   nil)
```

Los nodos del AST se agrupan en seis categorías según su papel en el árbol: estructura del workbook, expresiones de celda, navegación de fila, cuantificación existencial, expresiones cross-sheet dinámicas y formato condicional con posicionamiento relativo.

4.2.1 Estructura del workbook

Los nodos de estructura representan los contenedores del libro. Son los nodos de más alto nivel del árbol: el workbook agrupa hojas, las hojas agrupan regiones y las regiones agrupan tablas. Su jerarquía refleja directamente la estructura del documento de salida.

4.2.2 Expresiones de celda

Las expresiones de celda son los nodos que el backend traduce a representaciones de fórmula en el formato de salida. Abarcan tanto referencias a columnas de la misma tabla o de otra (`xl-expr-column-ref`, `xl-table-range`) como operaciones de agregación (`COUNTIF`, `COUNTA`, `SUM`) y literales de texto. Gracias a esta representación, el

Cuadro 4.1: Nodos de estructura del workbook.

Nodo	Atributos	Papel
<code>xl-workbook</code>	<code>name</code> , <code>sheets</code>	Raíz del árbol. <code>name</code> es el nombre del archivo de salida; <code>sheets</code> es la lista de hojas.
<code>xl-sheet</code>	<code>name</code> , <code>regions</code> , <code>fixed-expressions</code>	Una hoja del libro. Contiene regiones horizontales y expresiones en posición relativa fuera de las tablas.
<code>xl-region</code>	<code>tables</code>	Agrupar tablas colocadas una junto a otra en la misma hoja, separadas por una columna en blanco.
<code>xl-table</code>	<code>id</code> , <code>col-names</code> , <code>headers</code> , <code>contenido</code> , <code>computed</code> , <code>style-rules</code> , <code>cell-height</code> , <code>cell-width</code>	Plantilla de tabla. <code>col-names</code> enumera los símbolos DSL de columna; <code>computed</code> asocia columnas a expresiones de fórmula; <code>style-rules</code> almacena las reglas de coloreado condicional.

programador escribe (`countif rango criterio`) en lugar de componer manualmente la cadena de fórmula con coordenadas absolutas.

4.2.3 Navegación de fila y control de visibilidad

Ciertos cálculos necesitan referenciar la fila anterior o siguiente de la misma tabla—por ejemplo, acumular un valor incremental o mostrar una transición—o bien producir una celda visualmente vacía bajo alguna condición. Estos nodos encapsulan esa lógica sin exponer índices de fila al programador del DSL.

4.2.4 Cuantificación existencial

La cuantificación existencial es el mecanismo del DSL para expresar condiciones de la forma “existe alguna fila que cumpla ciertos criterios”. Mantenerla como un nodo AST propio, y no como una expresión booleana explícita, permite al backend generar la comprobación más adecuada según el contexto: una fórmula `SUMPRODUCT` vectorial en el backend Excel, u otra implementación para un backend diferente.

4.2.5 Expresiones cross-sheet dinámicas

Cuando un libro contiene múltiples hojas con estructura homogénea—por ejemplo, una hoja por grupo o aula—es necesario agregar valores a través de ellas. A diferencia de

`xl-expr-cross-sheet-ref`, que referencia una hoja con nombre fijo, los nodos de esta sección trabajan con una *variable de hoja* resuelta en tiempo de compilación mediante el entorno dinámico `*sheet-env*`, declarado por `collect-over`.

4.2.6 Formato condicional y posicionamiento

Estos nodos permiten expresar el coloreado condicional de celdas y la colocación de expresiones en posiciones relativas a las tablas. La separación en nodos propios garantiza que el DSL no exponga coordenadas absolutas: el backend resuelve las posiciones finales en función del tamaño real de los datos.

4.3 Definición de tablas: `def-table`

La forma central del DSL para definir datos tabulares es `def-table`. Produce una *plantilla* de tabla reutilizable: una descripción abstracta de las columnas, sus cabeceras, las fórmulas que se calculan a partir de ellas y la regla de coloreado que se aplica sobre sus filas. La plantilla no contiene datos; se instancia con datos concretos al ensamblar el workbook (sección 4.7).

Listing 4.2: Sintaxis completa de `def-table`.

```
(def-table nombre-tabla
  ((col1 "Cabecera1") (col2 "Cabecera2") ...) ; columnas
  :cell-height N          ; filas por fila logica (default 1)
  :cell-width N           ; columnas por columna logica (default
    1)
  :computed                ; formulas derivadas (opcional)
    ((colN (expresion-dsl)))
  :render                  ; regla de coloreado (opcional)
    (conditional-rendering
     :condition (expresion-booleana)
     :target-columns (col1 col2 ...)))
```

Cada par `(col "Cabecera")` declara un nombre de columna DSL (símbolo) y el texto de su encabezado visual. El orden de las columnas es el orden físico en el documento de salida.

La clave `:computed` asocia columnas a expresiones DSL que el compilador traduce a fórmulas. Los valores iniciales de esas columnas en los datos de entrada se ignoran: la celda siempre mostrará el resultado de la fórmula calculada.

La clave `:render` acepta una forma `conditional-rendering` que especifica bajo qué condición se colorean determinadas columnas. El mecanismo concreto lo decide el backend.

El listado siguiente define la tabla de profesores del caso de uso principal, con columna calculada y regla de coloreado condicional:

Listing 4.3: Tabla de profesores con columna `computed` y regla de coloreado.

```
(def-table profesores-table
```

```

((nombre "") (grado "") (defensas ""))
:cell-height 1
:cell-width 1
:computed
  ;; Cuenta cuantas veces aparece este profesor en los
  ;; cinco roles
  ((defensas (countif (trange defensas-table tutor
                              secretario)
                      (col nombre))))
:render
  (conditional-rendering
   :condition
     (exists (r :rows-of defensas-table)
      :self-in (tutor oponente presidente vocal
                secretario)
      :matching (dia hora))
   :target-columns (nombre)))

```

La variable `r` en la forma `exists` es un alias documental de la fila iterada, análogo a un alias de tabla en SQL. La regla de coloreado declara que la celda `nombre` debe resaltarse cuando el profesor participa en alguna defensa con conflicto de horario; el mecanismo lo decide el backend.

4.4 Expresiones de celda

El DSL ofrece formas simbólicas para construir expresiones de celda sin escribir cadenas de fórmula directamente. El compilador resuelve los nombres de columna a sus coordenadas reales, los rangos a sus límites absolutos y las operaciones a su sintaxis en el formato de salida; el programador solo declara la intención.

La tabla 4.7 recoge las formas disponibles.

La forma `trange` produce *referencias cross-table*: permite que una expresión en `profesores-table` cite columnas de `defensas-table` sin que el programador conozca las coordenadas reales. El compilador las resuelve en función del orden de declaración de las tablas en la región.

Listing 4.4: Expresiones DSL y sus fórmulas generadas (backend Excel).

```

;; (col nombre) en profesores-table -> columna K, fila
;; relativa
(col nombre)                ;; -> K4   (en la fila 4)

;; (trange defensas-table tutor secretario) -> rango absoluto
;; de los cinco roles
(trange defensas-table tutor secretario)  ;; -> $B$4:$F$6

;; La formula COUNTIF resultante
(countif (trange defensas-table tutor secretario) (col nombre
))

```



```
;; -> COUNTIF($B$4:$F$6, K4)
```

4.5 El cuantificador existencial **exists**

El cuantificador **exists** permite declarar condiciones de *existencia* sobre filas de una tabla. Preguntas del tipo “existe otra defensa con el mismo horario” o “aparece este profesor en alguna defensa con conflicto” son muy frecuentes en problemas de planificación, pero expresarlas mediante disyunciones y bucles explícitos produce código verboso y difícil de leer. Con **exists** el programador declara la intención semántica a alto nivel; el backend se encarga de generar la comprobación correcta.

El cuantificador existe en dos variantes según el dominio de búsqueda.

4.5.1 Variante same-table: **:all-rows**

Esta variante busca otra fila dentro de la *misma* tabla que satisfaga condiciones de coincidencia de valores y solapamiento en columnas de rol. Se usa para detectar si una fila del propio conjunto tiene algún conflicto con otra fila del mismo conjunto; por ejemplo, dos defensas programadas en el mismo día y hora con al menos un integrante de tribunal en común.

Listing 4.5: Cuantificador exists sobre la tabla actual.

```
(exists (r :all-rows)
  :matching (dia hora) ; mismos valores en
    ambas filas
  :any-overlap (tutor oponente presidente vocal secretario))
  ; algun valor
    compartido en estos
    slots
```

La semántica es: “existe otra fila de esta tabla (distinta de la actual) cuya columna **dia** y cuya columna **hora** coinciden con los de la fila actual, y que además comparte al menos un valor en las columnas de rol”.

En el backend Excel, esta expresión se traduce a una fórmula SUMPRODUCT que multiplica tres factores: coincidencia en las columnas de agrupación, exclusión de la fila actual y solapamiento en al menos una columna de rol.

4.5.2 Variante cross-table: **:rows-of**

Esta variante busca en las filas de *otra* tabla, relacionando el valor de la celda actual con columnas de esa tabla. Se usa cuando el elemento evaluado (p.ej. un profesor) y los datos de planificación (p.ej. las defensas) residen en tablas distintas.

Listing 4.6: Cuantificador exists sobre otra tabla (cross-table).

```
(exists (r :rows-of defensas-table)
  :self-in (tutor oponente presidente vocal secretario))
```

```

; el valor de la celda actual aparece en
  alguno de estos slots
:matching (dia hora))
; ese slot tiene mas de una defensa programada

```

La semántica es: “existe alguna fila de `defensas-table` en la que el valor de la celda actual (el nombre del profesor) aparece en una de las cinco columnas de rol, y cuyo par (`dia`, `hora`) tiene más de una defensa programada”.

En ambas variantes el alias de fila es puramente documental: el backend genera directamente una fórmula vectorial y no evalúa el símbolo como variable de programa. Su papel es análogo al de un alias de tabla en una sentencia SQL.

4.6 Formato condicional declarativo

El coloreado de celdas es una herramienta habitual en los documentos de planificación: resaltar en rojo los profesores con conflicto permite detectar problemas de un vistazo. El DSL ofrece la forma `conditional-rendering` para declarar ese coloreado sin escribir fórmulas ni llamadas a la API de formato.

Listing 4.7: Sintaxis de conditional-rendering.

```

(conditional-rendering
 :condition      (expresion-booleana)
 :target-columns (col1 col2 ...))

```

Cuando el nodo condición es una instancia de `xl-expr-exists`, el backend del caso de demostración produce una `FormulaRule` de Excel incrustada en el archivo `.xlsx`, en lugar de colorear celdas estáticamente. Esta regla persiste en el libro y se reevalúa automáticamente cada vez que el usuario edita los datos, sin necesidad de volver a ejecutar el DSL.

La decisión de usar `FormulaRule` en lugar de colorear celdas directamente es exclusiva del backend: el programador del DSL no indica qué mecanismo de formato se utilizará.

4.7 Ensamblaje del workbook

Una vez definidas las plantillas de tabla, el programador ensambla el workbook con tres macros: `libro` crea el nodo raíz del AST con el nombre del archivo de salida; `hoja` agrupa tablas y expresiones fijas en una pestaña del libro; `tabla` instancia una plantilla con datos concretos.

Listing 4.8: Ensamblaje del workbook con libro, hoja y tabla.

```

(libro nombre-libro
 :filename "Archivo.xlsx"
 :hojas (list
  (hoja "NombreHoja"
   (tabla plantilla1 :data *DATOS-1*)
   (tabla plantilla2 :data *DATOS-2*)

```

```
:fixed-expressions
  (((nav ...) (expresion))
   ((nav ...) (expresion))))))
```

Cuando una hoja recibe más de una tabla, el generador las coloca en la misma región horizontal, separadas por una columna en blanco. Este diseño permite que las referencias cross-table (`trange`) se resuelvan con coordenadas absolutas correctas.

La clave `:fixed-expressions` acepta pares de (posición, expresión). La posición se especifica con `nav`; su valor se recalcula según el tamaño real de los datos, de modo que los totales de pie de tabla se desplazan automáticamente si se añaden más filas.

4.8 Posicionamiento relativo: `nav`

Las expresiones fijas de una hoja no pueden tener coordenadas absolutas codificadas, pues esas coordenadas dependen de cuántas filas de datos tenga cada tabla en cada ejecución. La forma `nav` resuelve esto: calcula la posición de una celda como un *ancla* fija en una tabla más una secuencia de pasos direccionales.

Las anclas disponibles son `ultima-fila` y `primera-fila`. A partir del ancla se encadenan pasos `abajo`, `arriba`, `izquierda` y `derecha` con su cantidad:

Listing 4.9: Sintaxis de `nav` y anclas de tabla.

```
;; Una fila debajo de la ultima fila de datos
(nav (ultima-fila tabla-id col-name) abajo 1)

;; Una fila debajo y dos columnas a la derecha
(nav (ultima-fila tabla-id col-name) abajo 1 derecha 2)

;; Una fila encima de la primera fila de datos
(nav (primera-fila tabla-id col-name) arriba 1)
```

El compilador resuelve el ancla a la coordenada correcta en tiempo de generación y produce una referencia absoluta que siempre apunta al lugar correcto, independientemente del número de filas de datos.

4.9 Generación y ejecución

Las dos últimas formas del DSL cierran el ciclo de compilación: `xl-generate` recorre el árbol AST y produce el programa de construcción del libro; `xl-run-generated` ejecuta ese programa y crea el archivo de salida.

Listing 4.10: Compilación del AST y ejecución.

```
;; Recorre el AST y genera el programa de construccion del
  libro
(xl-generate horario-tesis "horario-tesis.py")

;; Ejecuta el programa generado y crea el archivo .xlsx
```

```
(xl-run-generated "horario-tesis.py")
```

4.10 Ejemplo completo: Horario de Defensas de Tesis

El caso de uso principal del DSL es la generación automática de un horario de defensas de tesis con dos tablas relacionadas. La primera tabla, `defensas-table`, registra cada defensa con su tribunal (cinco roles: tutor, oponente, presidente, vocal y secretario), su slot horario y el local. La segunda tabla, `profesores-table`, enumera los profesores con su grado académico y el número de defensas en que participan; sus filas se colorean en rojo cuando el profesor tiene un conflicto de horario.

El listado siguiente es el programa DSL completo; en él se observan todas las construcciones descritas en el capítulo.

Listing 4.11: Programa DSL completo: Horario de Defensas de Tesis.

```
(load "dsl-directo.lisp")
(load "data-tutorial.lisp")    ; *DATOS-DEFENSAS*, *DATOS-
                                PROFESORES*

;; -- TABLA 1: defensas
-----
;; Nueve columnas: cinco de rol (tutor...secretario) mas dia,
    hora, local.
;; Las cinco columnas de rol son contiguas para poder
    referenciarlas como
;; rango con (trange defensas-table tutor secretario).
(def-table defensas-table
  ((estudiante "") (tutor "") (oponente "") (presidente "")
   (vocal "") (secretario "") (dia "") (hora "") (local ""))
  :cell-height 1
  :cell-width 1)

;; -- TABLA 2: profesores
-----
;; La columna "defensas" cuenta apariciones del nombre en los
    cinco roles.
;; El formato condicional marca en rojo a quienes tienen
    conflicto.
(def-table profesores-table
  ((nombre "") (grado "") (defensas ""))
  :cell-height 1
  :cell-width 1
  :computed
  ((defensas (countif (trange defensas-table tutor
                              secretario)
                      (col nombre))))
  :render
  (conditional-rendering
   :condition
```

```

    (exists (r :rows-of defensas-table)
      :self-in (tutor oponente presidente vocal
        secretario)
      :matching (dia hora))
    :target-columns (nombre)))

;; -- WORKBOOK
-----
(libro horario-tesis
 :filename "Horario_Tesis.xlsx"
 :hojas (list
  (hoja "Defensas"
    (tabla defensas-table :data *DATOS-DEFENSAS*)
    (tabla profesores-table :data *DATOS-PROFESORES*)
    :fixed-expressions
      (((nav (ultima-fila defensas-table estudiante) abajo
        1)
        (str "Total defensas:"))
        ((nav (ultima-fila defensas-table estudiante) abajo
          1 derecha 1)
          (counta (trange defensas-table estudiante)))
        ((nav (ultima-fila profesores-table nombre) abajo 1)
          (str "Total profesores:"))
        ((nav (ultima-fila profesores-table nombre) abajo 1
          derecha 2)
          (sum-range (trange profesores-table defensas)))))))

(xl-generate horario-tesis "horario-tesis.py")
(xl-run-generated "horario-tesis.py")

```

Los datos de entrada contienen tres defensas y seis profesores. Dos de las defensas comparten el slot Lunes 10:00 con roles solapados. El programa genera `Horario_Tesis.xlsx` con la siguiente estructura:

- Columnas de `defensas-table` con tres filas de datos.
- Columna separadora en blanco de la región.
- Columnas de `profesores-table` con seis filas de datos y la columna `Defensas` calculada mediante `COUNTIF`.
- Fila de totales debajo de cada tabla con posicionamiento dinámico.
- Regla de formato condicional aplicada a la columna `nombre`: los profesores que participan en al menos una defensa cuyo slot tiene más de una defensa programada aparecen resaltados en rojo.

La regla de coloreado implementa exactamente la semántica del `exists` declarado en el DSL. Al ser una `FormulaRule` incrustada en el archivo, cualquier edición posterior de los datos provoca una reevaluación automática del coloreado, sin necesidad de volver a ejecutar el DSL.

Cuadro 4.2: Nodos de expresión de celda.

Nodo	Atributos	Papel
xl-expr-column-ref	name, context	Referencia a la columna name en la fila actual. Se resuelve a la coordenada de esa columna en el formato de salida.
xl-range	from-col, to-col	Rango de columnas contiguas de la tabla actual.
xl-table-range	table-id, from-col, to-col	Rango en una tabla específica de la región (referencia cross-table). Lo produce la forma trange .
xl-expr-countif	count-range, criteria	Fórmula de conteo condicional sobre un rango.
xl-expr-counta	count-range	Conteo de celdas no vacías.
xl-expr-sum	count-range	Suma sobre un rango.
xl-expr-string	value	Literal de texto; produce una cadena fija en la celda.
xl-expr-if	test, then, else	Condicional de tres ramas.
xl-expr-equals	a, b	Igualdad entre dos expresiones.
xl-expr-and, xl-expr-or	a, b	Conjunción y disyunción lógica.
xl-expr-concat	a, b	Concatenación de texto.
xl-expr-table-col-ref	table-id, name, context	Referencia a una columna de una tabla específica de la región. Evita la ambigüedad cuando dos tablas comparten el mismo nombre de columna.
xl-expr-param-ref	name	Referencia a un parámetro de instancia almacenado en celda auxiliar.
xl-expr-different	a, b	Desigualdad entre dos expresiones.
xl-expr-gt, xl-expr-lt	a, b	Comparación estricta: mayor que y menor que.
xl-expr-gte, xl-expr-lte	a, b	Comparación no estricta: mayor o igual y menor o igual.
xl-expr-add, xl-expr-subtract	a, b	Suma y resta aritmética.
xl-expr-multiply, xl-expr-divide	a, b	Multipliación y división aritmética.

Cuadro 4.3: Nodos de navegación de fila y control de visibilidad.

Nodo	Atributos	Papel
<code>xl-expr-previous-row</code>	<code>expr</code>	Evalúa <code>expr</code> referenciando la columna en la fila anterior de la tabla.
<code>xl-expr-next-row</code>	<code>expr</code>	Evalúa <code>expr</code> referenciando la columna en la fila siguiente de la tabla.
<code>xl-expr-first-row</code>	—	Produce <code>TRUE</code> cuando la fila en evaluación es la primera fila de datos.
<code>xl-expr-non-empty</code>	<code>expr</code>	Produce <code>TRUE</code> cuando <code>expr</code> no es vacía ni cero.
<code>xl-expr-show-nothing</code>	—	Produce una cadena vacía: la celda queda visualmente vacía.

Cuadro 4.4: Nodos de cuantificación existencial.

Nodo	Atributos	Papel
<code>xl-expr-exists</code>	<code>bind-var</code> , <code>domain</code> , <code>match-keys</code> , <code>overlap-cols</code> , <code>self-in-cols</code>	Cuantificador existencial estructurado. Declara que debe existir una fila del dominio que cumpla las condiciones expresadas por los slots semánticos. El backend decide el mecanismo de comprobación según el tipo de dominio.
<code>xl-domain-rows</code>	<code>table-id</code>	Dominio de filas. <code>table-id = nil</code> indica la tabla actual; un símbolo identifica una tabla específica de la región (dominio cross-table).

Cuadro 4.5: Nodos de expresión cross-sheet dinámica.

Nodo	Atributos	Papel
<code>xl-expr-cross-sheet-sheet</code>	<code>sheet</code> , <code>cell-template</code>	Referencia a una celda en otra hoja con nombre fijo. <code>cell-template</code> puede incluir <code>\${row-num}</code> para fila dinámica.
<code>xl-expr-cross-cell</code>	<code>sheet</code> , <code>xcol</code> , <code>row</code>	Referencia a una celda en la hoja ligada a <code>sheet</code> en <code>*sheet-env*</code> .
<code>xl-expr-source-row</code>	<code>table-id</code> , <code>offset</code>	Fila de una tabla origen mapeada desde la fila lógica actual. <code>offset</code> selecciona la subcelda dentro de una fila compuesta.
<code>xl-expr-sheet-id</code>	<code>sheet</code>	Identificador canónico de la hoja ligada a <code>sheet</code> en <code>*sheet-env*</code> .
<code>xl-expr-collect-over</code>	<code>groups</code> , <code>sheet-var</code> , <code>body</code>	Itera <code>body</code> sobre cada hoja de <code>groups</code> , ligando <code>sheet-var</code> , y concatena los resultados como lista separada por comas.

Cuadro 4.6: Nodos de formato condicional y posicionamiento relativo.

Nodo	Atributos	Papel
<code>xl-style-rule</code>	<code>rule-condition</code> , <code>target-columns</code>	Regla de coloreado condicional. <code>rule-condition</code> es un nodo de expresión booleana; <code>target-columns</code> enumera las columnas que se colorean cuando la condición es verdadera.
<code>xl-anchor</code>	<code>table-id</code> , <code>col-name</code> , <code>anchor-type</code>	Punto de referencia dentro de una tabla. <code>anchor-type</code> es <code>:ultima-fila</code> o <code>:primera-fila</code> .
<code>xl-nav</code>	<code>anchor</code> , <code>steps</code>	Posición calculada como un ancla más una secuencia de pasos direccionales (arriba, abajo, izquierda, derecha).
<code>xl-sheet-fixed-exprpos</code> , <code>expr</code>		Expresión colocada en una posición relativa de la hoja. <code>pos</code> es un nodo <code>xl-nav</code> .

Cuadro 4.7: Formas de expresión del DSL y su significado.

Forma DSL	Significado
<code>(col nombre)</code>	Valor de la columna <code>nombre</code> en la fila actual.
<code>(trange tabla-id desde hasta)</code>	Rango que abarca desde la columna <code>desde</code> hasta la columna <code>hasta</code> de <code>tabla-id</code> , para todas las filas de datos.
<code>(countif rango criterio)</code>	Conteo condicional sobre un rango.
<code>(counta rango)</code>	Número de celdas no vacías en el rango.
<code>(sum-range rango)</code>	Suma de los valores del rango.
<code>(str "texto")</code>	Literal de texto fijo.
<code>(_equals a b)</code>	Igualdad entre dos expresiones.
<code>(_and a b)</code> , <code>(_or a b)</code>	Conjunción y disyunción.
<code>(nav ancla dir n ...)</code>	Posición relativa: ancla más pasos direccionales.

Capítulo 5

El backend Excel: generación de archivos `.xlsx`

El DSL produce un árbol de sintaxis abstracta independiente de cualquier sistema de salida. Este capítulo describe el backend implementado como demostración: la traducción de ese AST a un programa Python que, mediante la biblioteca `openpyxl`, construye un archivo `.xlsx` con fórmulas y formato condicional persistente. La misma arquitectura —función genérica CLOS que despacha sobre el tipo de nodo— puede emplearse para implementar backends alternativos sin modificar el DSL ni el AST.

5.1 Arquitectura del backend

El backend está organizado en dos archivos con responsabilidades separadas:

`code-generation-utils.lisp`. Funciones auxiliares, variables dinámicas de configuración y la función de entrada principal. Incluye: `col->letter` (conversión de número de columna a letra Excel), `build-col-map` (construcción del mapa símbolo DSL $\rightarrow$ letra), `xl-generate` y `xl-run-generated`.

`generate-code-direct.lisp`. Los métodos CLOS de la función genérica `compile-excel-formula` y los métodos del generador de código Python. Este archivo es el único lugar del sistema donde pueden aparecer letras de columna Excel, cadenas de fórmula y llamadas a `openpyxl`.

El principio de separación queda codificado en la arquitectura: `ast-def.lisp` declara *qué existe*, `dsl-directo.lisp` declara *cómo se escribe*, y `generate-code-direct.lisp` declara *a qué se traduce en este backend*. Un backend alternativo reemplazaría únicamente los dos últimos archivos sin tocar el AST ni el DSL.

5.2 Resolución de coordenadas

5.2.1 Conversión de número de columna a letra Excel

La función `col->letter` convierte un entero n (la posición ordinal de una columna, con base 1) a la letra o letras de columna correspondientes en Excel: columna 1 $\rightarrow$ A, columna 26 $\rightarrow$ Z, columna 27 $\rightarrow$ AA, etc.

Listing 5.1: Función col->letter.

```
(defun col->letter (n)
  (with-output-to-string (s)
    (loop while (> n 0)
      do (multiple-value-bind (q r) (floor (1- n) 26)
          (setf n q)
          (write-char (code-char (+ 65 r)) s))))))
```

El algoritmo es una conversión de base 10 a base 26 con el ajuste habitual $\lfloor (n-1)/26 \rfloor$, $((n-1) \bmod 26) + 65$ que mapea los residuos a los caracteres ASCII de las letras mayúsculas.

5.2.2 Construcción del mapa de columnas

Para cada tabla de la región se construye un *mapa de columnas*: una lista de asociación que relaciona cada símbolo DSL de columna con su letra Excel absoluta. La función `build-col-map` construye el mapa de una tabla sola, asumiendo que empieza en la columna 1. La función `build-table-col-maps` construye el mapa global de toda la región, teniendo en cuenta el desplazamiento de cada tabla dentro de la región:

Listing 5.2: Construcción del mapa de columnas de una región.

```
(defun build-col-map (tbl)
  (loop for name in (col-names tbl)
    for i from 1
    collect (cons name (col->letter i))))

(defun build-table-col-maps (tables offsets)
  (loop for tbl in tables
    for offset in offsets
    collect (cons (id tbl)
      (loop for name in (col-names tbl)
        for i from 1
        collect (cons name
          (col->letter (+
            offset i))))))))
```

El resultado se almacena en la variable dinámica `*table-col-maps*`, disponible durante toda la compilación de la región.

5.2.3 Cálculo de rangos de fila

Cada tabla tiene una primera fila de datos (`first-row`, configurable, por defecto 4) y una última fila que depende del número de filas de datos y de `cell-height`:

$$last-row = first-row + |datos| \times cell-height - 1 \quad (5.1)$$

Estos valores se almacenan en la variable dinámica `*table-data-rows*` como una lista de asociación (`table-id . (first-row . last-row)`), lo que permite que los nodos

`xl-table-range` usen los rangos de fila correctos para cada tabla aunque la compilación se realice dentro del contexto de una tabla diferente.

5.3 La función genérica `compile-excel-formula`

El método central del backend es la función genérica `compile-excel-formula`, que toma un nodo AST y devuelve la cadena de fórmula Excel correspondiente. Su firma es:

Listing 5.3: Declaración de `compile-excel-formula`.

```
(defgeneric compile-excel-formula
  (expr col-map data-names row-num first-row last-row)
  (:documentation
    "Compila un nodo AST de expresion a string de formula
    Excel"))
```

Los parámetros son:

expr El nodo AST a compilar.

col-map El mapa de columnas de la tabla actual (`sym → letra`).

data-names Lista de símbolos DSL de la tabla actual (para `VLOOKUP`).

row-num Número de fila Excel que se está compilando (para referencias relativas).

first-row Primera fila de datos de la tabla actual.

last-row Última fila de datos de la tabla actual.

El despacho múltiple de CLOS garantiza que, para cada tipo de nodo, se ejecuta el método correcto sin ningún `cond` centralizado. Añadir un nuevo tipo de expresión al lenguaje sólo requiere:

1. Definir una nueva clase con `defclass*` en `ast-def.lisp`.
2. Definir un macro en `dsl-directo.lisp` que construya el nodo.
3. Definir un `defmethod` de `compile-excel-formula` en `generate-code-direct.lisp`.

Ningún código existente necesita modificarse.

5.3.1 Métodos para expresiones simples

Listing 5.4: Métodos `compile-excel-formula` para expresiones simples.

```
;; Referencia a columna de la tabla actual: "K4"
(defmethod compile-excel-formula
  ((e clase-xl-expr-column-ref) col-map data-names
   row-num first-row last-row)
  (let* ((letter (cdr (assoc (name e) col-map)))
        (ctx (context e)))
    (if (typep ctx 'clase-xl-expr-previous-row)
        (format nil "~a~a" letter (1- row-num))
        (format nil "~a~a" letter row-num))))
```

```
;; Comparacion de igualdad: "K4=B4"
(defmethod compile-excel-formula
  ((e clase-xl-expr-equals) col-map data-names
   row-num first-row last-row)
  (format nil "~a=~a"
    (compile-excel-formula (a e) col-map data-names
                           row-num first-row last-row)
    (compile-excel-formula (b e) col-map data-names
                           row-num first-row last-row)
  ))

;; Literal de texto: "\"Total\""
(defmethod compile-excel-formula
  ((e clase-xl-expr-string) col-map data-names
   row-num first-row last-row)
  (format nil "~s" (value e)))
```

5.3.2 Métodos para rangos y agregados

Listing 5.5: Método para xl-table-range.

```
;; Rango absoluto de columnas de una tabla especifica de la
;; region.
;; Usa *table-data-rows* para obtener los limites exactos de
;; esa tabla.
(defmethod compile-excel-formula
  ((e clase-xl-table-range) col-map data-names
   row-num first-row last-row)
  (let* ((tmap (cdr (assoc (table-id e) *table-col-
    maps*)))
        (from-letter (cdr (assoc (name (from-col e)) tmap)))
        (to-letter (if (to-col e)
                        (cdr (assoc (name (to-col e)) tmap)
                        )
                        from-letter)))
        (tbl-rows (cdr (assoc (table-id e) *table-data-
    rows*)))
        (act-first (if tbl-rows (car tbl-rows) first-row))
        (act-last (if tbl-rows (cdr tbl-rows) last-row)))
    (format nil "$~a$~a:$~a$~a"
      from-letter act-first to-letter act-last)))
```

La clave técnica de este método es que consulta `*table-data-rows*` con el identificador de la *tabla referenciada*, no con el de la tabla actual. Así, `(trange defensas-table tutor secretario)` compila al rango de `defensas-table` aunque se esté compilando una fórmula de `profesores-table`.

5.4 Generación de formato condicional

El formato condicional es el aspecto técnicamente más elaborado de la generación de código. El backend recibe un nodo `x1-expr-exists` y produce una fórmula SUMPRODUCT que implementa la semántica del cuantificador existencial declarado.

5.4.1 Variante same-table: SUMPRODUCT con exclusión de fila actual

Cuando `exists` usa `:all-rows`, el backend genera una fórmula SUMPRODUCT que, para cada fila, multiplica tres factores:

1. Coincidencia en las columnas de agrupación (`:matching`).
2. Exclusión de la fila actual (para no comparar una fila consigo misma).
3. Solapamiento en al menos una columna de rol (`:any-overlap`).

Para una regla con `:matching` (`dia hora`) y `:any-overlap` (`tutor ... secretario`), la fórmula generada para la fila 4 es:

Listing 5.6: Fórmula Excel CF para la variante same-table (fila 4).

```
=SUMPRODUCT (
  ($G$4:$G$6=$G4) *           -- dia coincide en ambas filas
  ($H$4:$H$6=$H4) *           -- hora coincide en ambas
    filas
  (ROW($B$4:$B$6)<>ROW($B4)) * -- excluir la fila actual
  (($B$4:$B$6=$B4)+($C$4:$C$6=$C4)+...+($F$4:$F$6=$F4))
) > 0
```

Las referencias con columna fija y fila relativa (p.ej. `$G4`) hacen que la fórmula se evalúe correctamente para cada fila del rango: al desplazarse de la fila 4 a la fila 5, las referencias de “fila actual” avanzan automáticamente. El resultado de SUMPRODUCT es el número de filas del dominio que satisfacen las tres condiciones; la comparación `> 0` convierte ese conteo en un valor booleano para la regla de formato condicional.

5.4.2 Variante cross-table: SUMPRODUCT con COUNTIFS

Cuando `exists` usa `:rows-of`, el backend genera una fórmula SUMPRODUCT que combina dos condiciones:

1. Que el valor de la celda actual aparezca en al menos una columna de rol de la tabla referenciada (`:self-in`).
2. Que el slot de tiempo de esa fila tenga más de una defensa programada (`:matching` con COUNTIFS).

Para la regla de `profesores-table` con `:self-in` (`tutor ... secretario`) y `:matching` (`dia hora`), la fórmula generada para la celda K4 es:

Listing 5.7: Fórmula Excel CF para la variante cross-table (celda K4).

```
=SUMPRODUCT (
  (($B$4:$B$6=$K4)+($C$4:$C$6=$K4)+($D$4:$D$6=$K4)+
```


para eliminar espacios extras y reemplazarlos con comas, produciendo una lista de grupos separada por comas.

Para el horario de la facultad con 17 grupos, el método genera una fórmula que agrega la ocupación del Aula 1 en el turno 3 del lunes a través de las 17 hojas del workbook, sin que el programador haya escrito ningún nombre de hoja ni letra de columna.

5.6 Generación del programa Python

El generador principal recorre el árbol del workbook y emite un programa Python que usa `openpyxl` para construir el archivo `.xlsx`. El programa generado:

1. Crea un workbook `openpyxl`.
2. Para cada hoja, crea una hoja de trabajo y la configura.
3. Para cada tabla de cada región, escribe las cabeceras y los datos.
4. Para cada columna calculada, escribe la fórmula en cada fila de datos (compilada por `compile-excel-formula` en tiempo de generación del Python).
5. Para cada regla de estilo condicional, crea una `FormulaRule` con la fórmula `SUMPRODUCT` generada y la añade al rango correspondiente.
6. Para cada expresión fija, resuelve la posición `nav` y escribe la expresión compilada en la celda correspondiente.
7. Guarda el workbook en el archivo `.xlsx` configurado.

El listado siguiente muestra el fragmento del programa Python generado para la columna calculada `defensas` de `profesores-table`:

Listing 5.9: Fragmento del programa Python generado para la columna `computed`.

```
# Columna: defensas (profesores-table)
ws["M4"] = "=COUNTIF($B$4:$F$6,K4)"
ws["M5"] = "=COUNTIF($B$4:$F$6,K5)"
ws["M6"] = "=COUNTIF($B$4:$F$6,K6)"
ws["M7"] = "=COUNTIF($B$4:$F$6,K7)"
ws["M8"] = "=COUNTIF($B$4:$F$6,K8)"
ws["M9"] = "=COUNTIF($B$4:$F$6,K9)"

# FormulaRule para profesores-table, columna nombre
from openpyxl.formatting.rule import FormulaRule
from openpyxl.styles import PatternFill
red_fill = PatternFill(start_color="FF6666",
                       end_color="FF6666", fill_type="solid")
formula = ("SUMPRODUCT(((B$4:B$6=$K4)+(C$4:C$6=$K4)+("
           "($D$4:$D$6=$K4)+($E$4:$E$6=$K4)+($F$4:$F$6=$K4))*"
           " "
           "(COUNTIFS($G$4:$G$6,$G$4:$G$6,$H$4:$H$6,$H$4:$H$6"
           ")>1))>0)")
ws.conditional_formatting.add(
    "K4:K9",
```



```
FormulaRule(formula=[formula], fill=red_fill))
```

Las fórmulas de las celdas están calculadas en tiempo de generación del Python: el programa Python que se ejecuta sólo asigna cadenas de texto a las celdas. Las referencias relativas dentro de las fórmulas las evalúa Excel al abrir el archivo. La `FormulaRule`, en cambio, se reevalua en Excel dinámicamente cada vez que cambian los datos.

Capítulo 6

Detalles de implementación

Este capítulo describe los aspectos internos del sistema: la organización de los archivos con sus responsabilidades, los macros que implementan el DSL, el patrón de extensibilidad del backend y la validación del sistema sobre los tres casos de uso presentados en el capítulo 3.

6.1 Organización en cuatro archivos

El sistema está organizado en cuatro archivos Lisp con responsabilidades estrictamente separadas. La dependencia entre ellos es unívoca: cada archivo carga únicamente al que le precede.

Esta separación tiene consecuencias prácticas importantes: es posible cargar `ast-def.lisp` y construir un AST sin haber cargado el backend, lo que facilita las pruebas de la sintaxis y la semántica del DSL de forma independiente.

6.2 Macros del DSL

Los macros del DSL se clasifican en tres grupos según su papel en la jerarquía del AST.

6.2.1 Macros de expresión

Los macros de expresión construyen nodos de la familia de expresiones de celda. Todos son equivalencias directas: reciben los argumentos del programador y los pasan al constructor CLOS correspondiente, haciendo el quoting adecuado para los símbolos del dominio.

Listing 6.1: Macros de expresión representativos.

```
;; (col nombre) -> xl-expr-column-ref con nombre quoteado
(defmacro col (name &optional context)
  (if context
    '(xl-expr-column-ref :name ',name :context ,context)
    '(xl-expr-column-ref :name ',name :context nil)))

;; (trange tabla desde) / (trange tabla desde hasta)
```

Cuadro 6.1: Archivos del sistema y sus responsabilidades.

Archivo	Responsabilidad
<code>codigo-tesis.lisp</code>	Utilidades generales: <code>symb</code> , <code>mkstr</code> , <code>keywd</code> , <code>ppme</code> y la macro <code>defclass*</code> . No conoce ningún concepto del dominio de hojas de cálculo.
<code>ast-def.lisp</code>	Definición de todas las clases del AST con <code>defclass*</code> . Backend-agnóstico: no contiene ninguna letra de columna Excel, cadena de fórmula ni referencia a ninguna biblioteca de salida.
<code>code-generation-utils.lisp</code> <code>generate-code-direct.lisp</code>	Backend Excel/Python. Único lugar del sistema donde viven letras de columna, cadenas de fórmula, estructuras <code>openpyxl</code> y conocimiento de la disposición concreta del workbook. El primero contiene las utilidades y variables dinámicas; el segundo, los métodos <code>compile-excel-formula</code> .
<code>dsl-directo.lisp</code>	Macros del DSL: <code>def-table</code> , <code>tabla</code> , <code>hoja</code> , <code>hoja-v</code> , <code>libro</code> , <code>exists</code> , <code>conditional-rendering</code> , <code>nav</code> , <code>col</code> , <code>tcol</code> , <code>trange</code> , <code>countif</code> , <code>counta</code> , <code>sum-range</code> , <code>str</code> , <code>concat</code> , <code>time-add</code> , y demás. No contiene lógica de generación de código.

```
;; -> xl-table-range con los dos extremos como xl-expr-table-
    col-ref
(defmacro trange (table-id from &optional to)
  '(xl-table-range
    :table-id ',table-id
    :from-col (xl-expr-table-col-ref :table-id ',table-id
                                     :name ',from :context
                                     nil)
    :to-col   ,(if to
                  '(xl-expr-table-col-ref :table-id ',table-id
                                           :name ',to :
                                           context nil)
                  nil)))

;; (exists (var :rows-of tabla) :self-in (...) :matching
  (...))
;; -> xl-expr-exists con domain cross-table
(defmacro exists ((var &rest domain-spec)
                  &key matching any-overlap self-in)
  (let ((domain-type (first domain-spec)))
    (ecase domain-type
      (:all-rows
```

```

      '(xl-expr-exists :bind-var      ',var
                      :domain        (xl-domain-rows :table
                                                    -id nil)
                      :match-keys    ',matching
                      :overlap-cols  ',any-overlap
                      :self-in-cols  nil))
    (:rows-of
     (let ((table-id (second domain-spec)))
       '(xl-expr-exists :bind-var      ',var
                       :domain        (xl-domain-rows
                                       :table-id ',table-id)
                       :match-keys    ',matching
                       :overlap-cols  ',any-overlap
                       :self-in-cols  ',self-in))))))

```

La regla de quoting de los macros es determinista: un símbolo que es nombre de columna o tabla en el dominio se quotea (`'`, `sym`); un símbolo que es una variable Lisp en scope no se quotea (`,`, `sym`). Esta distinción es explícita en todos los macros.

6.2.2 Macros de estructura

Los macros `def-table`, `hoja`, `hoja-v` y `libro` construyen los nodos de la jerarquía del workbook.

`def-table` es el más complejo: genera una función que construye un `xl-table` con sus slots rellenos. Los `:inst-params` declarados se añaden a la firma de esa función, lo que permite instanciar la misma plantilla de tabla con valores de parámetro distintos:

Listing 6.2: Expansión simplificada de `def-table`.

```

(def-table aulas-dia-table
  ((dia "") (aula1 "") ...)
  :inst-params (dia)
  :computed ((aula1 (collect-over *grupos-all* (g) ...))))

;; Genera la funcion:
(defun aulas-dia-table (&key data params dia)
  (xl-table :id 'aulas-dia-table
            :col-names '(dia aula1 ...)
            :computed (list (cons 'aula1 (collect-over ...)))
            ...))

;; Que luego se instancia con:
(tabla aulas-dia-table :dia lun :data *datos-aulas-lunes*)
;; -> (aulas-dia-table :data *datos-aulas-lunes* :dia 'lun)

```

6.2.3 El macro `nav` y el sistema de posicionamiento

El macro `nav` convierte una secuencia plana de pares dirección–cantidad en una lista de cons-celdas:

Listing 6.3: Implementación del macro `nav`.

```
(defmacro nav (anchor &rest steps)
  (let ((step-pairs
        (loop for (dir n) on steps by #'cddr
              collect '(cons ',dir ,n))))
    `(xl-nav :anchor ,anchor
             :steps (list ,@step-pairs))))

;; Ejemplo: (nav (ultima-fila t c) abajo 1 derecha 2)
;; Expande a:
;; (xl-nav :anchor (xl-anchor :table-id 't :col-name 'c
;;                             :anchor-type :ultima-fila)
;;         :steps (list (cons 'abajo 1) (cons 'derecha 2)))
```

El backend resuelve los pasos en tiempo de generación: dado el número de fila de la última celda de datos, suma o resta las cantidades según la dirección y produce la coordenada Excel absoluta.

6.3 Extensibilidad del sistema

La extensibilidad es una consecuencia directa del uso de CLOS y de la separación en archivos. Considrese el ejemplo de añadir soporte para la función `AVERAGEIF` de Excel, que actualmente no está en el lenguaje.

Paso 1 — Nuevo nodo AST (en `ast-def.lisp`):

```
(defclass* xl-expr-averageif () (average-range criteria))
```

Paso 2 — Nuevo macro (en `dsl-directo.lisp`):

```
(defmacro averageif (range-expr criteria)
  `(xl-expr-averageif :average-range ,range-expr
                     :criteria ,criteria))
```

Paso 3 — Nuevo método (en `generate-code-direct.lisp`):

```
(defmethod compile-excel-formula
  ((e clase-xl-expr-averageif) col-map data-names
   row-num first-row last-row)
  (let ((range-str (compile-excel-formula
                    (average-range e) col-map data-names
                    row-num first-row last-row))
        (criteria (compile-excel-formula
                  (criteria e) col-map data-names
                  row-num first-row last-row)))
    (cons range-str criteria)))
```

```
(format nil "AVERAGEIF(~a,~a)" range-str criteria)))
```

Ningún código existente necesita modificarse. El macro `defclass*` garantiza que la nueva clase tiene exactamente la misma interfaz que las demás.

6.4 Validación sobre los casos de uso

Los tres casos de uso descritos en el capítulo 3 se validaron ejecutando el DSL y verificando manualmente el archivo `.xlsx` generado.

6.4.1 Horario de Defensas de Tesis

El programa DSL del listado 4.11 genera `Horario_Tesis.xlsx` con la estructura esperada. La verificación confirmó:

- La columna `Defensas` de `profesores-table` muestra los valores correctos: Piad = 2, García = 2, López = 3, Torres = 3, Ruiz = 2, Soto = 2.
- Los profesores López y Torres, que participan en las dos defensas del slot `Lunes 10:00`, aparecen resaltados en rojo.
- Los totales en la fila inferior a las tablas son correctos: “Total defensas: 3” y “Total profesores: 6” con las sumas esperadas.
- La `FormulaRule` persiste en el archivo: al cambiar manualmente la hora de la tercera defensa de `10:00` a `14:00`, el resaltado de López y Torres desaparece automáticamente en Excel.

6.4.2 Horario Docente de la Facultad

El programa `ast-facultad.lisp` genera `Horario_Facultad.xlsx` con diecisiete hojas de grupo (Ciencia de la Computación en cuatro años, Matemática en cuatro años y Ciencia de Datos en cuatro años) más una hoja de ocupación de aulas.

La verificación confirmó:

- Las columnas `asignadas` y `faltan` de `stats-table` calculan correctamente para cada grupo a partir del contenido de `turno-table`.
- La hoja de aulas muestra en cada celda los grupos que ocupan cada aula en cada turno, con la lista de grupos separada por comas producida por las fórmulas `SUBSTITUTE(TRIM(...))`.
- Las celdas compuestas de `turno-table` (`cell-height = 2`) se generan correctamente: la subfila superior muestra la asignatura y la inferior el aula.

Este caso ejercita, además, la característica de `inst-params`: la tabla `aulas-dia-table` se define una sola vez y se instancia una vez por día de la semana, con `:dia lun`, `:dia mar`, etc.

6.4.3 Discusión comparativa

Para ilustrar el beneficio del DSL, se estima la cantidad de código necesaria para implementar el caso de defensas de forma imperativa directa en Python con `openpyxl`, en comparación con el programa DSL.

La versión imperativa directa requiere calcular manualmente las letras de columna, construir las cadenas de fórmula `COUNTIF` con interpolación de texto, y ensamblar la fórmula `SUMPRODUCT` con referencias absolutas calculadas a partir del número de filas. Una implementación representativa ocupa aproximadamente 80–100 líneas de Python.

El programa DSL equivalente (listado 4.11) ocupa 40 líneas, de las cuales la mayor parte son la declaración de datos (`def-table`, `libro`) y la expresión de intención (`exists`, `conditional-rendering`). Ninguna línea contiene una letra de columna, un índice de fila o una cadena de fórmula.

Más importante que el número de líneas es que el programa DSL es resistente a cambios estructurales: añadir una columna a `defensas-table` sólo requiere añadir un par (`nombre` "Cabecera") a la lista de columnas; el compilador recalcula automáticamente todas las letras y rangos. En la versión imperativa, el mismo cambio requiere actualizar manualmente todas las referencias a columnas afectadas.

Conclusiones

El trabajo alcanzó los objetivos propuestos. Se diseñó e implementó un lenguaje de dominio específico en Common Lisp para la descripción declarativa de estructuras tabulares de planificación, y se validó sobre tres casos reales de la Facultad de Matemática y Computación de la Universidad de La Habana mediante un backend que produce archivos `.xlsx`.

Las conclusiones principales son las siguientes.

1. **La separación entre descripción semántica y backend de generación es alcanzable con un DSL interno en Lisp.** El sistema permite declarar tablas relacionadas, columnas calculadas, cuantificadores existenciales y reglas de coloreado condicional sin hacer referencia a ningún detalle concreto del sistema de salida: ningún índice de columna, ninguna cadena de fórmula, ninguna llamada a API alguna. El backend es el único punto del sistema donde viven esos detalles, y es intercambiable sin modificar el programa DSL.
2. **Las macros de Common Lisp son el mecanismo adecuado para implementar el DSL.** La homoiconicidad del lenguaje hace que los macros sean transformadores de código completamente naturales: cada forma del DSL es una lista anidada que el macro convierte en constructores CLOS, sin ningún parser externo.
3. **El despacho múltiple de CLOS ofrece extensibilidad sin modificar el código existente.** Añadir un nuevo tipo de expresión al lenguaje requiere únicamente tres acciones: definir la clase del nodo con `defclass*`, definir el macro de superficie y definir el método del backend. El patrón se replicó consistentemente en los más de veinte tipos de nodo implementados.
4. **El cuantificador `exists` captura correctamente las condiciones de conflicto de planificación.** Las dos variantes (`:all-rows` y `:rows-of`) cubren los patrones de conflicto más frecuentes en los documentos analizados. El backend genera fórmulas `SUMPRODUCT` que implementan esas semánticas de forma robusta y producen `FormulaRule` persistentes en el `.xlsx`.
5. **El posicionamiento relativo con `nav` elimina las coordenadas absolutas de los programas del DSL.** Totales, etiquetas y celdas de resumen se colocan siempre en la posición correcta independientemente del número de filas de datos, lo que hace al programa resiliente ante cambios en el tamaño de los conjuntos.
6. **El backend implementado genera archivos `.xlsx` con formato condicional**

activo. La decisión de usar `FormulaRule` en lugar de coloreado estático demuestra que la arquitectura es capaz de producir salidas con comportamiento dinámico: la hoja de cálculo sigue siendo funcional cuando el usuario la edita directamente, sin necesidad de volver a ejecutar el DSL. Un backend alternativo podría reproducir esta propiedad en su propio formato de salida.

Recomendaciones

A partir del trabajo realizado se identifican las siguientes líneas de extensión para investigaciones futuras.

1. **Soporte para más funciones de celda.** El DSL actual cubre `COUNTIF`, `COUNTA`, `SUM`, `IF`, `AVERAGEIF` y operaciones de tiempo. Funciones como `INDEX/MATCH`, `XLOOKUP`, `PIVOTBY` y las nuevas funciones de matrices dinámicas de Excel 365 podrían añadirse siguiendo exactamente el mismo patrón de extensión descrito en la sección 6.3.
2. **Backends alternativos.** La independencia del backend respecto al AST hace posible implementar generadores de código para otros formatos: ODS (LibreOffice Calc), CSV anotado, HTML con tablas, o incluso bases de datos SQL. La capa del DSL no requiere ningún cambio.
3. **Interfaz de usuario gráfica.** El AST es una estructura de datos de primera clase en Lisp. Una interfaz gráfica podría visualizar y editar el AST directamente, permitiendo a usuarios no programadores configurar las reglas de coloreado y los cálculos sin escribir código.
4. **Verificación de tipos del DSL.** El sistema actual no comprueba en tiempo de expansión de macros que los símbolos de columna referenciados en `exists` y `trange` existan en las tablas correspondientes. Un verificador de tipos a nivel del AST detectaría estos errores antes de la generación del código Python.
5. **Generación incremental.** El sistema genera el programa Python completo en cada ejecución. Un modo incremental que detecte qué tablas cambiaron y sólo regenere las hojas afectadas reduciría el tiempo de ciclo en workbooks con muchas hojas.
6. **Plantillas reutilizables.** Las plantillas de tabla definidas con `def-table` podrían exportarse a una biblioteca de plantillas estándar para los documentos más comunes de la facultad (horarios de docencia, defensas, actas de exámen), reduciendo el esfuerzo de configuración inicial para cada nuevo caso.

Bibliografía

- [1] M. Fowler. *Domain-Specific Languages*. Addison-Wesley Professional, 2010.
- [2] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson/Addison-Wesley, 2^a ed., 2006.
- [3] G. L. Steele. *Common Lisp: The Language*. Digital Press, 2^a ed., 1990.
- [4] P. Graham. *On Lisp: Advanced Techniques for Common Lisp*. Prentice-Hall, 1993.
- [5] P. Seibel. *Practical Common Lisp*. Apress, 2005.
- [6] S. E. Keene. *Object-Oriented Programming in Common Lisp: A Programmer's Guide to CLOS*. Addison-Wesley, 1989.
- [7] E. Gazoni y C. Clark. openpyxl — A Python library to read/write Excel 2010 xlsx/xlsm/xltx/xltm files. <https://openpyxl.readthedocs.io>, 2010–2024.
- [8] ISO/IEC. *ISO/IEC 29500: Information technology — Office Open XML File Formats*. International Organization for Standardization, 2008.
- [9] A. van Deursen, P. Klint y J. Visser. Domain-specific languages: an annotated bibliography. *ACM SIGPLAN Notices*, 35(6):26–36, 2000.
- [10] M. Mernik, J. Heering y A. M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005.
- [11] J. McCarthy. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4):184–195, 1960.
- [12] D. G. Bobrow, L. G. DeMichiel, R. P. Gabriel, S. E. Keene, G. Kiczales y D. A. Moon. Common Lisp Object System specification. *ACM SIGPLAN Notices*, 23(S):1–142, 1988.
- [13] G. Kiczales, J. des Rivières y D. G. Bobrow. *The Art of the Metaobject Protocol*. MIT Press, 1991.
- [14] Microsoft Corporation. *Excel for Microsoft 365 — Reference Guide*. Microsoft Docs, 2023.
- [15] H. Babaei, J. Karimpour y A. Hadidi. A survey of approaches for university course timetabling problem. *Computers & Industrial Engineering*, 86:43–59, 2015.
- [16] S. Abdennadher y M. Marte. University course timetabling using constraint handling rules. *Applied Artificial Intelligence*, 14(4):311–325, 2000.

- [17] T. Müller. ITC-2007 solver description: a hybrid approach. In *Proceedings of the 7th International Conference on the Practice and Theory of Automated Timetabling*, 2008.
- [18] S. Bollapragada y M. Garbiras. Scheduling commercials on broadcast television. *Operations Research*, 52(3):337–345, 2004.
- [19] J. A. Fernández y J. A. Larrañaga. A model for the television scheduling problem. *European Journal of Operational Research*, 191(2):576–588, 2008.
- [20] N. Pillay. A survey of school timetabling research. *Artificial Intelligence Review*, 45(4):517–543, 2016.
- [21] Coursedog Inc. *Coursedog Academic Operations Platform*. <https://coursedog.com>, 2021.
- [22] MEDIAGENIX. *WHATS'On — Broadcast and Media Management Software*. <https://www.mediagenix.tv>, 2024.
- [23] D. Matthews. *Broadcast Technology: A Practical Introduction*. Routledge, 2018.
- [24] MDN Web Docs Contributors. *JavaScript Frameworks and Libraries*. Mozilla Developer Network, 2024.
- [25] Society of Motion Picture and Television Engineers. *SMPTE ST 2021: Broadcast Exchange Format (BXF)*. SMPTE, 2021.
- [26] R. Portela Aguilera. Una aproximación a la generación automática de horarios para la Facultad de Matemática y Computación de la Universidad de La Habana. Trabajo de Diploma, MATCOM/UH, 2010.
- [27] E. Travieso Sosa. Modelo de restricciones para un sistema de generación automatizada de horarios. Trabajo de Diploma, MATCOM/UH, 2012.
- [28] G. Chang Alcover. Generación automática de horarios mediante técnicas de satisfacción de restricciones. Trabajo de Diploma, MATCOM/UH, 2014.
- [29] L. Fernández Llanos. API especializada para la evaluación incremental de restricciones en horarios universitarios. Trabajo de Diploma, MATCOM/UH, 2014.
- [30] D. Seguí Bonilla. Generación automática de horarios mediante metaheurísticas de búsqueda local. Trabajo de Diploma, MATCOM/UH, 2014.
- [31] J. Tomás López. Generación automática de horarios a partir de metaheurísticas de población. Trabajo de Diploma, MATCOM/UH, 2014.
- [32] L. Quintero Rojas. Sistema para la gestión y la generación automática de horarios. Trabajo de Diploma, MATCOM/UH, 2014.
- [33] M. Nardo González. Modelación matemática y herramienta web para resolver el problema de organizar defensas de tesis en la Facultad de Matemática y Computación. Trabajo de Diploma, MATCOM/UH, 2019.
- [34] A. Casteleiro Wong. Informatización de la gestión docente en la Facultad de Matemática y Computación. Trabajo de Diploma, MATCOM/UH, 2022.

- [35] V. R. Reddy et al. Graph coloring approach for university timetabling. *International Journal of Science and Research*, 2019.
- [36] H. Kim et al. Deep reinforcement learning for television broadcasting scheduling. *IEEE Transactions on Broadcasting*, 2021.
- [37] Horarium.ai. Plataforma de generación automática e interactiva de horarios mediante inteligencia artificial. Disponible en línea, 2024.
- [38] R. A. Cápiro Colomar. *LD MAG: Lenguaje para el diseño de microaplicaciones de gestión*. Trabajo de Diploma, Facultad de Matemática y Computación, Universidad de La Habana, 2025.
- [39] D. A. Watt. *Programming Language Design Concepts*. Wiley, 2004.
- [40] B. Liskov y S. Zilles. Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4):50–59, 1974.
- [41] M. L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4ª ed., 2015.